PRÁCTICA DE DOCKER EN VIRTUALBOX

Mi web y su vigilante

Enunciado de la práctica sobre una máquina virtual Ubuntu

Una máquina virtual, Docker dentro de ella y dos contenedores trabajando en equipo.

Contenido

1. Presentación de la práctica
2. Esquema de lo que vas a construir
3. Fase 1. Crear la máquina virtual
4. Fase 2. Instalar Ubuntu
5. Fase 3. Instalar Docker en Ubuntu
6. Fase 4. Preparar el proyecto
7. Fase 5. Crear la página web
8. Fase 6. Empaquetar la web con un Dockerfile
9. Fase 7. Unirlo todo con Docker Compose
10. Fase 8. Configurar el vigilante
11. Fase 9. La prueba de fuego: tirar la web
12. Fase 10. Página de estado, cambios y persistencia
13. Fase 11. Acceder desde el equipo anfitrión
14. Fase 12. Limpieza y entrega

1. Presentación de la práctica

Qué vas a construir

En esta práctica vas a crear una máquina virtual con Ubuntu en VirtualBox, vas a instalar Docker dentro de ella y, sobre esa base, vas a poner en marcha dos contenedores: uno con tu propia página web personal y otro con Uptime Kuma, una herramienta profesional de monitorización que comprueba cada pocos segundos si tu web está funcionando.

Cuando la web funcione, el panel la mostrará en verde, con una gráfica de su tiempo de respuesta. Cuando la detengas a propósito, el panel la marcará en rojo. Y cuando vuelvas a arrancarla, regresará al verde. Al final, abrirás la web y el panel desde el navegador de tu propio ordenador, atravesando la máquina virtual.

Qué vas a aprender

- **Virtualización:** crear y configurar una máquina virtual con VirtualBox.
- **Ubuntu:** instalar Ubuntu y trabajar con su terminal.
- **Instalación de Docker:** instalar Docker Engine desde el repositorio oficial.
- **Dockerfile e imágenes:** escribir un Dockerfile y construir una imagen propia.
- **Docker Compose:** arrancar varios contenedores a la vez con un solo archivo.
- **Red interna:** cómo dos contenedores se comunican entre sí por su nombre.
- **Volúmenes:** cómo conservar datos aunque un contenedor se elimine.
- **Reenvío de puertos:** cómo llegar desde tu ordenador a un servicio que está dentro de una máquina virtual.

Material necesario

- VirtualBox instalado en tu ordenador (versión 7 o posterior). Se descarga gratis desde virtualbox.org.
- La imagen ISO de Ubuntu Desktop 24.04 LTS, que se descarga gratis desde ubuntu.com.
- Al menos 8 GB de memoria RAM en tu ordenador (la máquina virtual usará 4 GB).
- Al menos 30 GB libres en el disco.
- Conexión a Internet.

Duración estimada

Entre dos horas y media y tres horas. Se recomienda dividirla en dos sesiones: en la primera, las fases 1 a 3 (máquina virtual, Ubuntu y Docker); en la segunda, el resto.

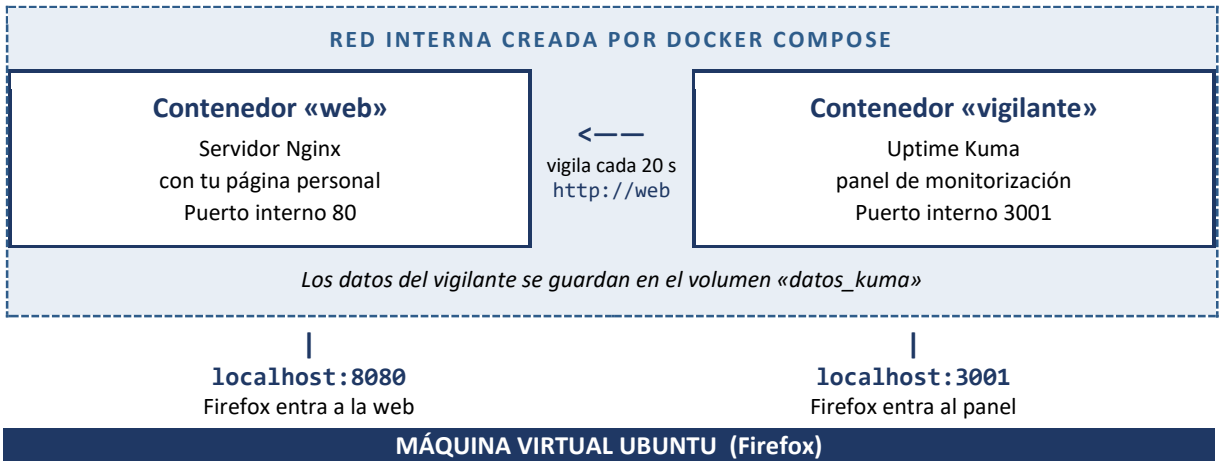
ANTES DE EMPEZAR

Comprueba que la virtualización está activada en la BIOS/UEFI de tu ordenador (suele llamarse Intel VT-x o AMD-V). Si no lo está, la máquina virtual no podrá arrancar o funcionará muy lenta.

Lee cada paso completo antes de hacerlo. Todos los pasos están explicados para que entiendas no solo qué haces, sino por qué lo haces.

2. Esquema de lo que vas a construir

Toda la práctica se desarrolla dentro de una máquina virtual Ubuntu. Dentro de ella, Docker ejecutará dos contenedores conectados por una red interna:



Y la máquina virtual, a su vez, se ejecuta en tu ordenador:

Nivel	Qué hay	Quién lo gestiona
Tu ordenador (anfitrión)	Tu sistema operativo y VirtualBox	Tú
Máquina virtual (invitado)	Ubuntu 24.04 con Docker Engine instalado	VirtualBox
Contenedores	La web y el vigilante	Docker

Las piezas, una a una

Pieza	Qué es	Para qué sirve
Contenedor «web»	Un servidor web Nginx con tu página dentro	Mostrar tu página en el navegador
Contenedor «vigilante»	La aplicación Uptime Kuma	Comprobar continuamente si la web responde
Red interna	Una red privada que crea Docker Compose	Que los contenedores se encuentren por su nombre
Volumen «datos_kuma»	Un espacio de almacenamiento fuera del contenedor	Guardar la configuración del vigilante
Puerto 8080	La entrada a la web desde Ubuntu	Abrir la web en http://localhost:8080
Puerto 3001	La entrada al panel desde Ubuntu	Abrir el panel en http://localhost:3001

Los archivos que vas a crear

Dentro de tu carpeta personal de Ubuntu crearás una carpeta con solo tres archivos:

```
~/mi-proyecto/
├── compose.yaml        (describe los dos contenedores)
└── web/
    ├── Dockerfile      (receta para crear la imagen de la web)
    └── index.html      (tu página web personal)
```

El símbolo `~` representa tu carpeta personal en Linux, por ejemplo `/home/alumno`.

3. Fase 1. Crear la máquina virtual

En esta fase vas a preparar en VirtualBox la máquina virtual donde se instalará Ubuntu. Una máquina virtual es un ordenador simulado: tiene su propia memoria, su propio disco y su propio sistema operativo, pero funciona dentro de una ventana de tu ordenador.

1

Descargar la imagen de Ubuntu

Descarga desde la web oficial de Ubuntu la imagen **Ubuntu Desktop 24.04 LTS**. Es un archivo con extensión `.iso` de unos 6 GB. Guárdalo en una carpeta que recuerdes, por ejemplo Descargas.

LTS significa «Long Term Support»: es una versión con soporte durante varios años, la más recomendable para trabajar.

2

Crear una máquina nueva

Abre VirtualBox y pulsa el botón **Nueva**. Se abrirá el asistente de creación. En la primera pantalla, rellena:

Campo	Valor	Explicación
Nombre	Ubuntu-Docker	Nombre con el que verás la máquina en VirtualBox
Imagen ISO	El archivo .iso descargado	El «disco de instalación» de Ubuntu
Tipo / Versión	Linux / Ubuntu (64-bit)	Normalmente se rellena solo al elegir la ISO
Omitir instalación desatendida	Marcada	Instalarás Ubuntu tú mismo, paso a paso

POR QUÉ OMITIR LA INSTALACIÓN DESATENDIDA

Si no marcas esta casilla, VirtualBox instala Ubuntu automáticamente. Es más rápido, pero a veces crea el usuario sin permisos de administrador, y entonces no podrías instalar Docker. Instalándolo tú controlas cada opción.

3

Asignar memoria y procesadores

En la pantalla de hardware, configura los siguientes valores. Son los mínimos para que Ubuntu y los dos contenedores funcionen con fluidez:

Campo	Valor
Memoria base	4096 MB (4 GB)
Procesadores	2

4 **Crear el disco duro virtual**

En la pantalla del disco, elige **Crear un disco duro virtual ahora** y asígnale un tamaño de **25 GB**. El disco virtual es un archivo en tu ordenador que irá creciendo a medida que se use, sin ocupar los 25 GB desde el principio.

5 **Terminar el asistente**

Revisa el resumen y pulsa **Terminar**. La máquina aparecerá en la lista de la izquierda, apagada.

6 **Ajustar la memoria de vídeo**

Selecciona la máquina, pulsa **Configuración > Pantalla** y sube la **Memoria de vídeo** a 128 MB. Así el escritorio de Ubuntu se moverá con más soltura. Pulsa Aceptar.

4. Fase 2. Instalar Ubuntu

Ahora vas a encender la máquina virtual e instalar Ubuntu en su disco virtual. El instalador es gráfico y sencillo. Los nombres de algunas pantallas pueden variar ligeramente según la versión exacta.

1 **Arrancar la máquina**

Selecciona `Ubuntu-Docker` y pulsa **Iniciar**. Se abrirá una ventana con la máquina virtual. Si aparece un menú negro, elige **Try or Install Ubuntu** y pulsa Intro.

2 **Recorrer el instalador**

Espera a que cargue el instalador y avanza por sus pantallas con estas opciones:

Pantalla	Opción que debes elegir
Idioma	Español
Accesibilidad	Dejarlo como está y pulsar Siguiente
Distribución del teclado	Español
Conexión a Internet	Usar conexión por cable (la que ofrece VirtualBox)
¿Qué desea hacer con Ubuntu?	Instalar Ubuntu
Tipo de instalación	Instalación interactiva
Aplicaciones	Selección por defecto
Software de terceros	Dejar las casillas sin marcar
¿Cómo desea instalar Ubuntu?	Borrar disco e instalar Ubuntu

> **TRANQUILIDAD CON «BORRAR DISCO»**
>
> La opción «Borrar disco e instalar Ubuntu» solo afecta al disco virtual de 25 GB que acabas de crear. Tu ordenador y tus archivos reales no se tocan.

3 **Crear tu usuario**

En la pantalla de cuenta, escribe tu nombre, el nombre del equipo y un nombre de usuario en minúsculas y sin espacios, y elige una contraseña.

Anota la contraseña. La necesitarás muchas veces: Ubuntu la pide cada vez que ejecutas un comando con `sudo` (es decir, con permisos de administrador).

4 **Zona horaria e instalación**

Elige **Madrid** como zona horaria, revisa el resumen y pulsa **Instalar**. La instalación tarda entre diez y veinte minutos.

5 **Reiniciar**

Al terminar, pulsa **Reiniciar ahora**. Si aparece un mensaje pidiendo retirar el medio de instalación, pulsa Intro: VirtualBox lo retira automáticamente.

6 **Iniciar sesión y actualizar el sistema**

Inicia sesión con tu usuario. Abre la terminal pulsando **Ctrl + Alt + T** y ejecuta los siguientes comandos para instalar las últimas actualizaciones. Al pedirte la contraseña, escríbela aunque no veas los caracteres en pantalla: es normal en Linux.

```
sudo apt update
sudo apt upgrade -y
```

7 **Instalar las Guest Additions (recomendado)**

Las Guest Additions son un complemento de VirtualBox que permite ajustar la pantalla al tamaño de la ventana y copiar y pegar texto entre tu ordenador y la máquina virtual. Esto último te ahorrará mucho tiempo al escribir los archivos de la práctica.

Primero instala las herramientas necesarias:

```
sudo apt install -y build-essential bzip2
```

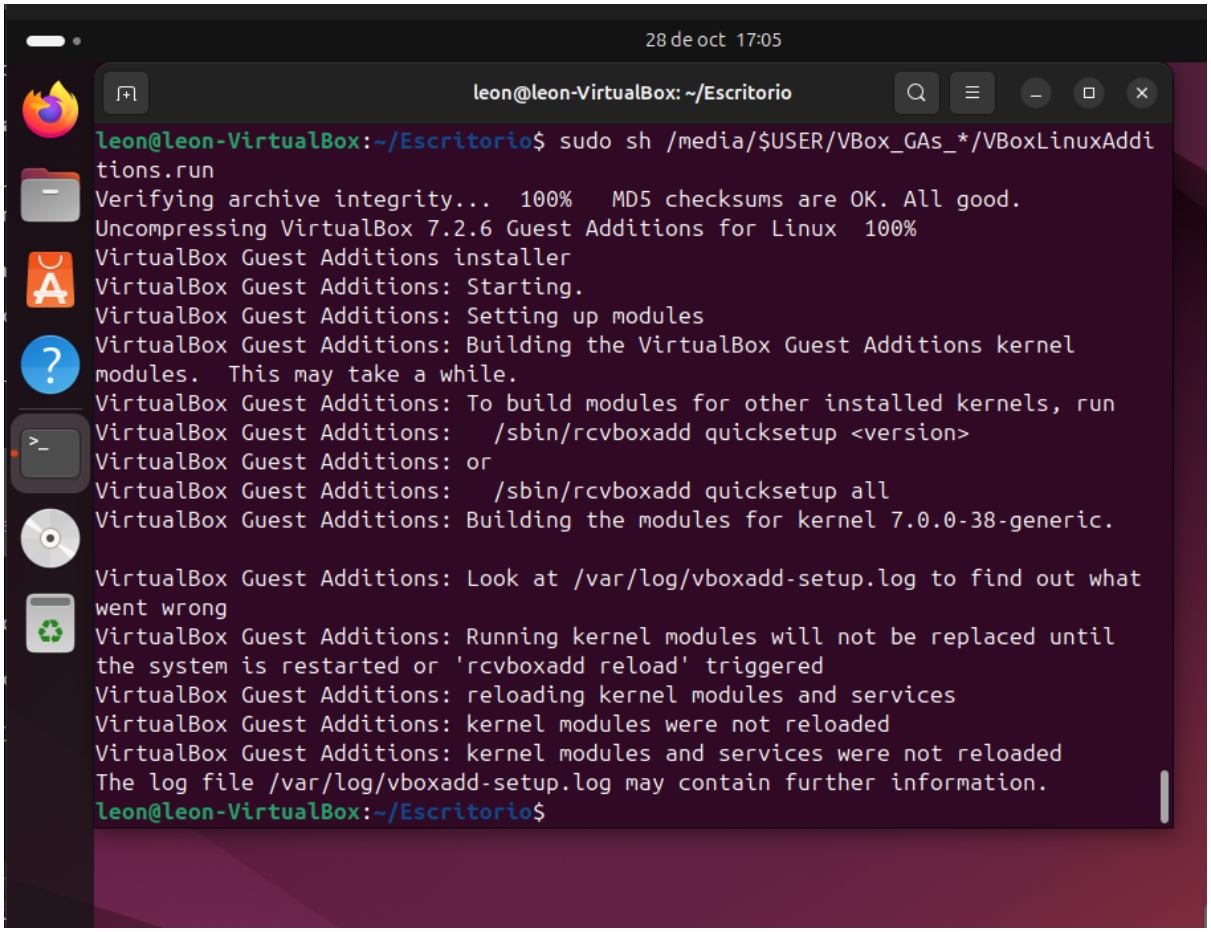
8 **Insertar y ejecutar el disco de las Guest Additions**

En el menú de la ventana de VirtualBox, elige **Dispositivos > Insertar imagen de CD de las «Guest Additions»**. Después, en la terminal, ejecuta el instalador y reinicia:

```
sudo sh /media/$USER/VBox_GAs_*/VBoxLinuxAdditions.run
sudo reboot
```

9 **Activar el portapapeles compartido**

Tras reiniciar, en el menú de VirtualBox elige **Dispositivos > Portapapeles compartido > Bidireccional**. A partir de ahora podrás copiar texto en tu ordenador y pegarlo en Ubuntu.


28 de oct 17:05
leon@leon-VirtualBox: ~/Escritorio
leon@leon-VirtualBox:~/Escritorio$ sudo sh /media/$USER/VBox_GAs_*/VBoxLinuxAdditions.run
Verifying archive integrity... 100% MD5 checksums are OK. All good.
Uncompressing VirtualBox 7.2.6 Guest Additions for Linux 100%
VirtualBox Guest Additions installer
VirtualBox Guest Additions: Starting.
VirtualBox Guest Additions: Setting up modules
VirtualBox Guest Additions: Building the VirtualBox Guest Additions kernel modules. This may take a while.
VirtualBox Guest Additions: To build modules for other installed kernels, run
VirtualBox Guest Additions: /sbin/rcvboxadd quicksetup <version>
VirtualBox Guest Additions: or
VirtualBox Guest Additions: /sbin/rcvboxadd quicksetup all
VirtualBox Guest Additions: Building the modules for kernel 7.0.0-38-generic.
VirtualBox Guest Additions: Look at /var/log/vboxadd-setup.log to find out what went wrong
VirtualBox Guest Additions: Running kernel modules will not be replaced until the system is restarted or 'rcvboxadd reload' triggered
VirtualBox Guest Additions: reloading kernel modules and services
VirtualBox Guest Additions: kernel modules were not reloaded
VirtualBox Guest Additions: kernel modules and services were not reloaded
The log file /var/log/vboxadd-setup.log may contain further information.
leon@leon-VirtualBox:~/Escritorio$

5. Fase 3. Instalar Docker en Ubuntu

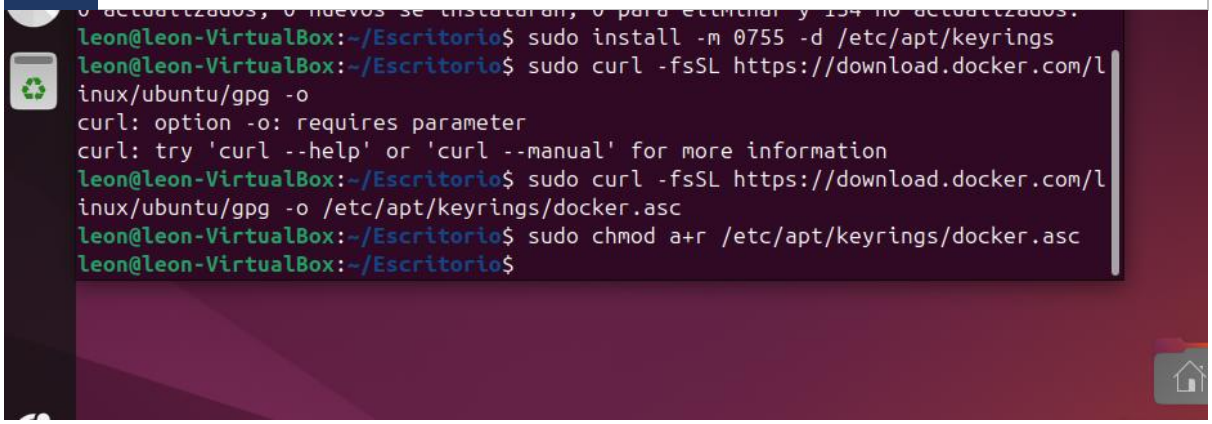
En Ubuntu no se usa Docker Desktop, sino Docker Engine: el motor de Docker instalado directamente en el sistema. Se instala desde el repositorio oficial de Docker para disponer siempre de la versión más reciente. Todos los comandos de esta fase se escriben en la terminal (Ctrl + Alt + T).

1

Preparar el sistema y la clave de seguridad

Estos comandos instalan dos herramientas necesarias y descargan la clave con la que Docker firma sus paquetes, para garantizar que son auténticos. Escríbelos uno a uno:

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
```



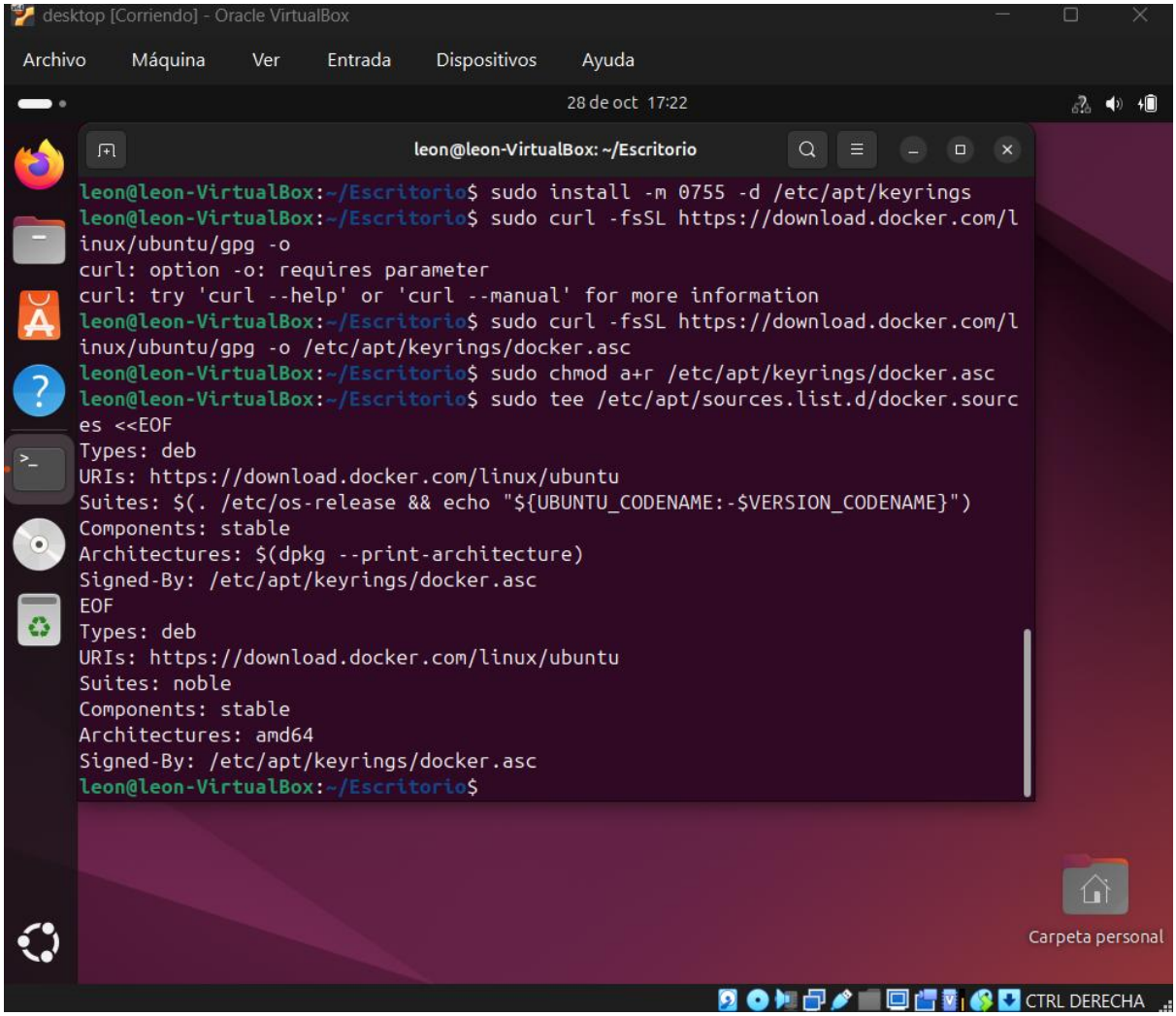
2

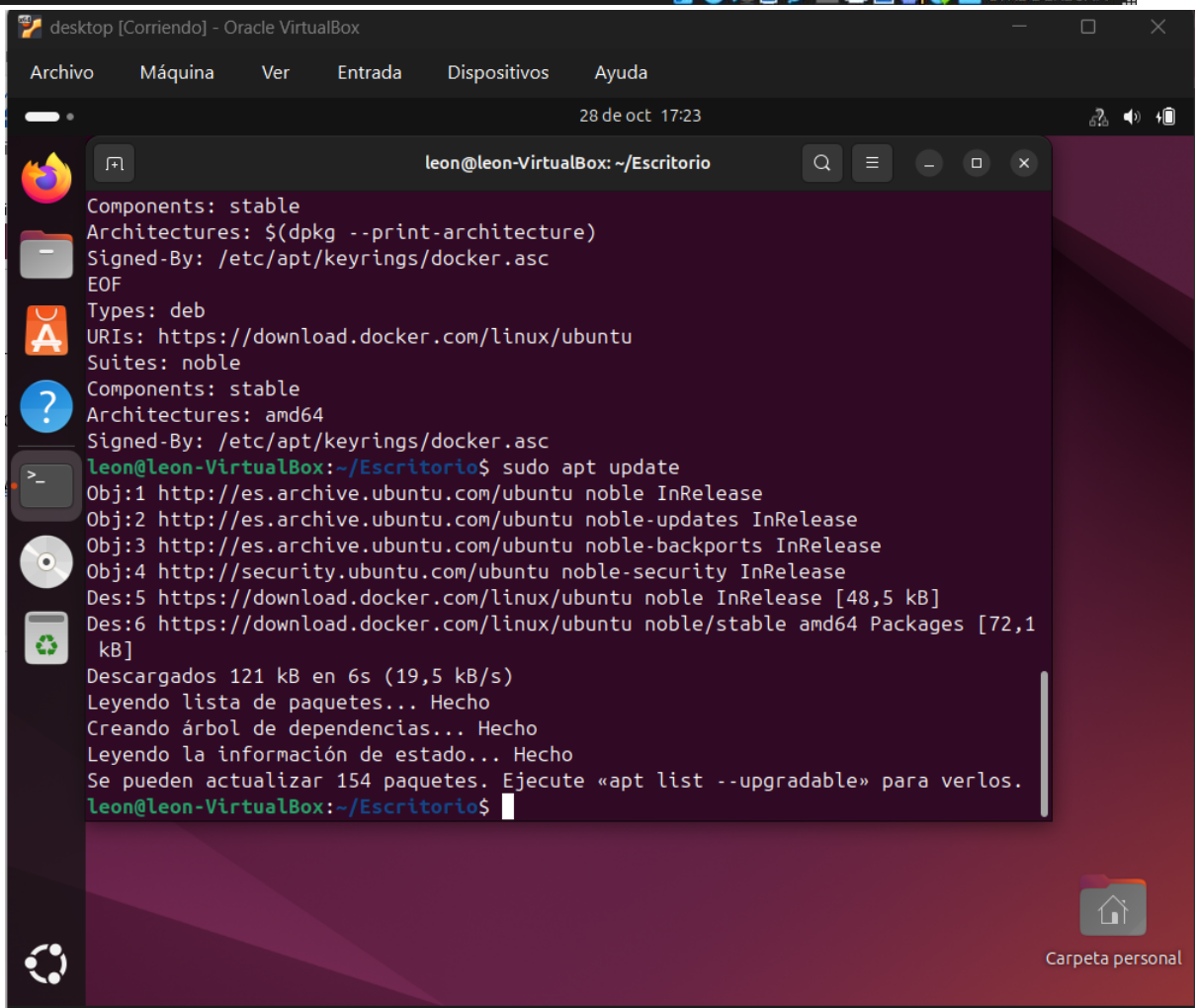
Añadir el repositorio oficial de Docker

Este bloque le indica a Ubuntu dónde encontrar los paquetes de Docker. Cópialo completo, desde la primera línea hasta la palabra EOF incluida, y pégalo en la terminal. Después, actualiza la lista de paquetes:

```
sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF

sudo apt update
```

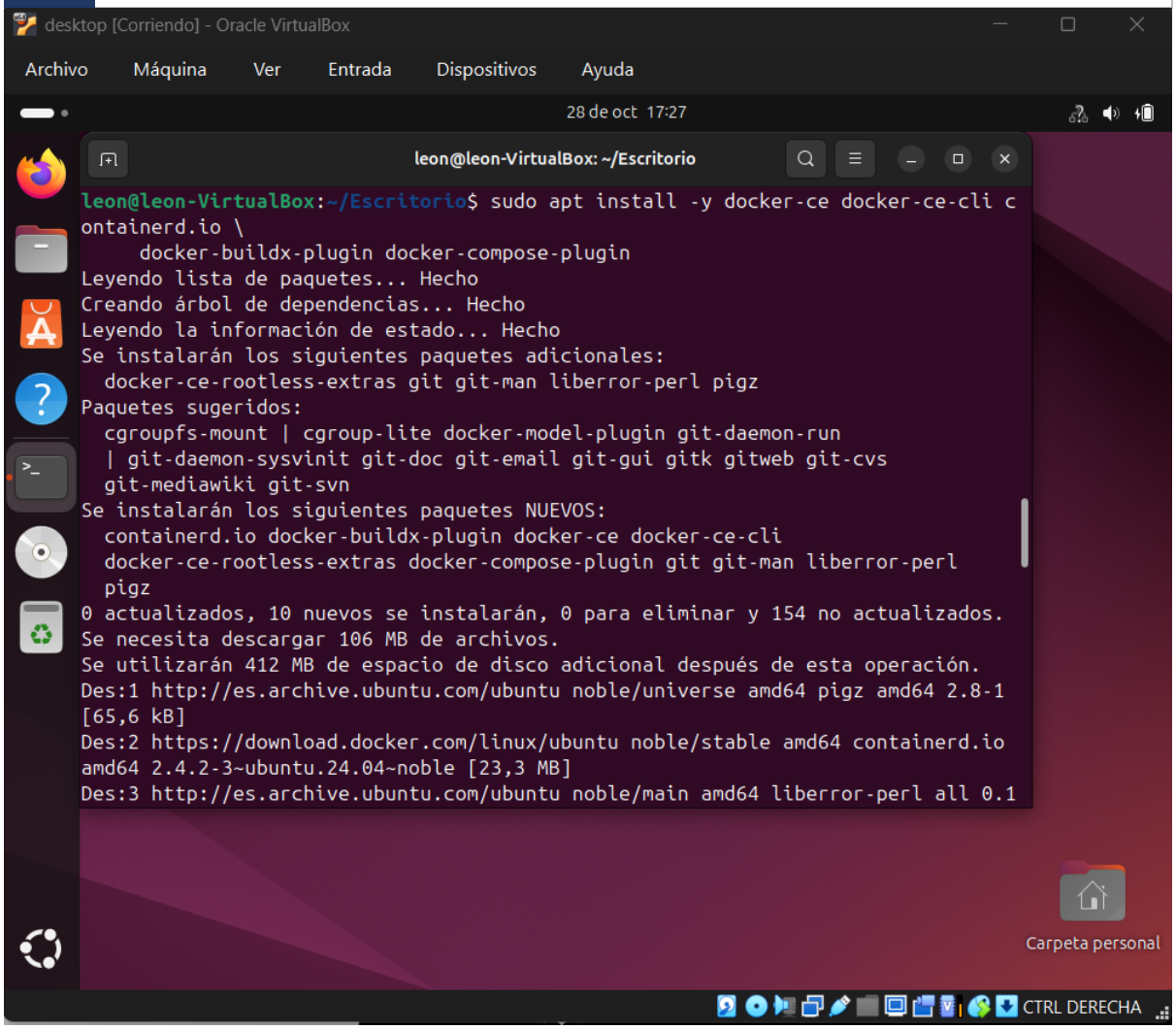

desktop [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
28 de oct 17:22
leon@leon-VirtualBox: ~/Escritorio
leon@leon-VirtualBox:~/Escritorio$ sudo install -m 0755 -d /etc/apt/keyrings
leon@leon-VirtualBox:~/Escritorio$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o
curl: option -o: requires parameter
curl: try 'curl --help' or 'curl --manual' for more information
leon@leon-VirtualBox:~/Escritorio$ sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
leon@leon-VirtualBox:~/Escritorio$ sudo chmod a+r /etc/apt/keyrings/docker.asc
leon@leon-VirtualBox:~/Escritorio$ sudo tee /etc/apt/sources.list.d/docker.sources <<EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
leon@leon-VirtualBox:~/Escritorio$
Carpeta personal
CTRL DERECHA


desktop [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
28 de oct 17:23
leon@leon-VirtualBox: ~/Escritorio
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
leon@leon-VirtualBox:~/Escritorio$ sudo apt update
Obj:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Obj:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Des:5 https://download.docker.com/linux/ubuntu noble InRelease [48,5 kB]
Des:6 https://download.docker.com/linux/ubuntu noble/stable amd64 Packages [72,1 kB]
Descargados 121 kB en 6s (19,5 kB/s)
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se pueden actualizar 154 paquetes. Ejecute «apt list --upgradable» para verlos.
leon@leon-VirtualBox:~/Escritorio$
Carpeta personal

3 Instalar Docker Engine

Instala el motor de Docker, su herramienta de comandos y los complementos para construir imágenes y usar Compose:

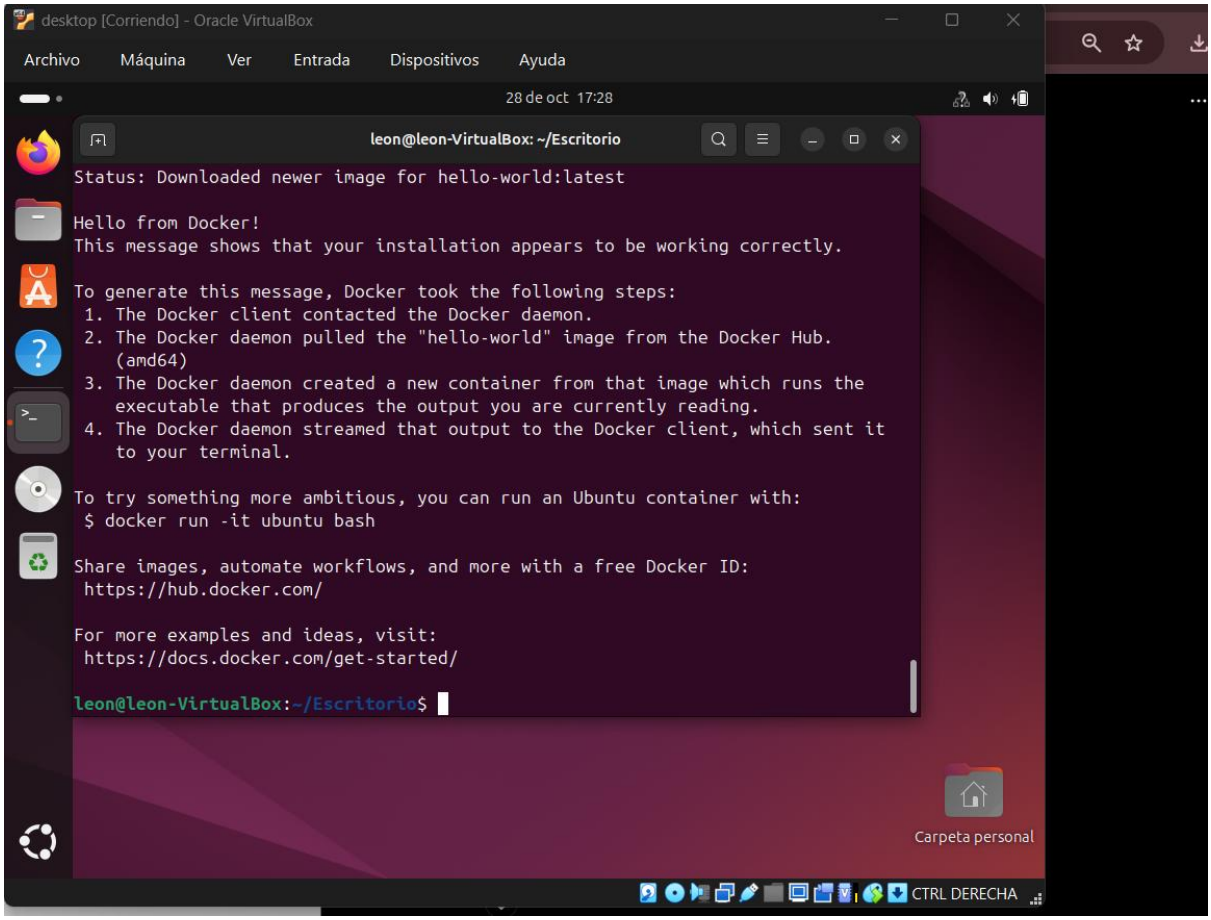
```
sudo apt install -y docker-ce docker-ce-cli containerd.io \
     docker-buildx-plugin docker-compose-plugin
```



4 Comprobar que funciona

Ejecuta el contenedor de prueba de Docker:

```
sudo docker run hello-world
```



QUÉ DEBE OCURRIR

Aparecerá un texto que empieza por «Hello from Docker!». Significa que Docker ha descargado una imagen de prueba, ha creado un contenedor y lo ha ejecutado correctamente.

5 Usar Docker sin escribir sudo

Para no tener que escribir `sudo` delante de cada comando de Docker, añade tu usuario al grupo `docker`:

```
sudo usermod -aG docker $USER
```

6 Cerrar sesión y volver a entrar

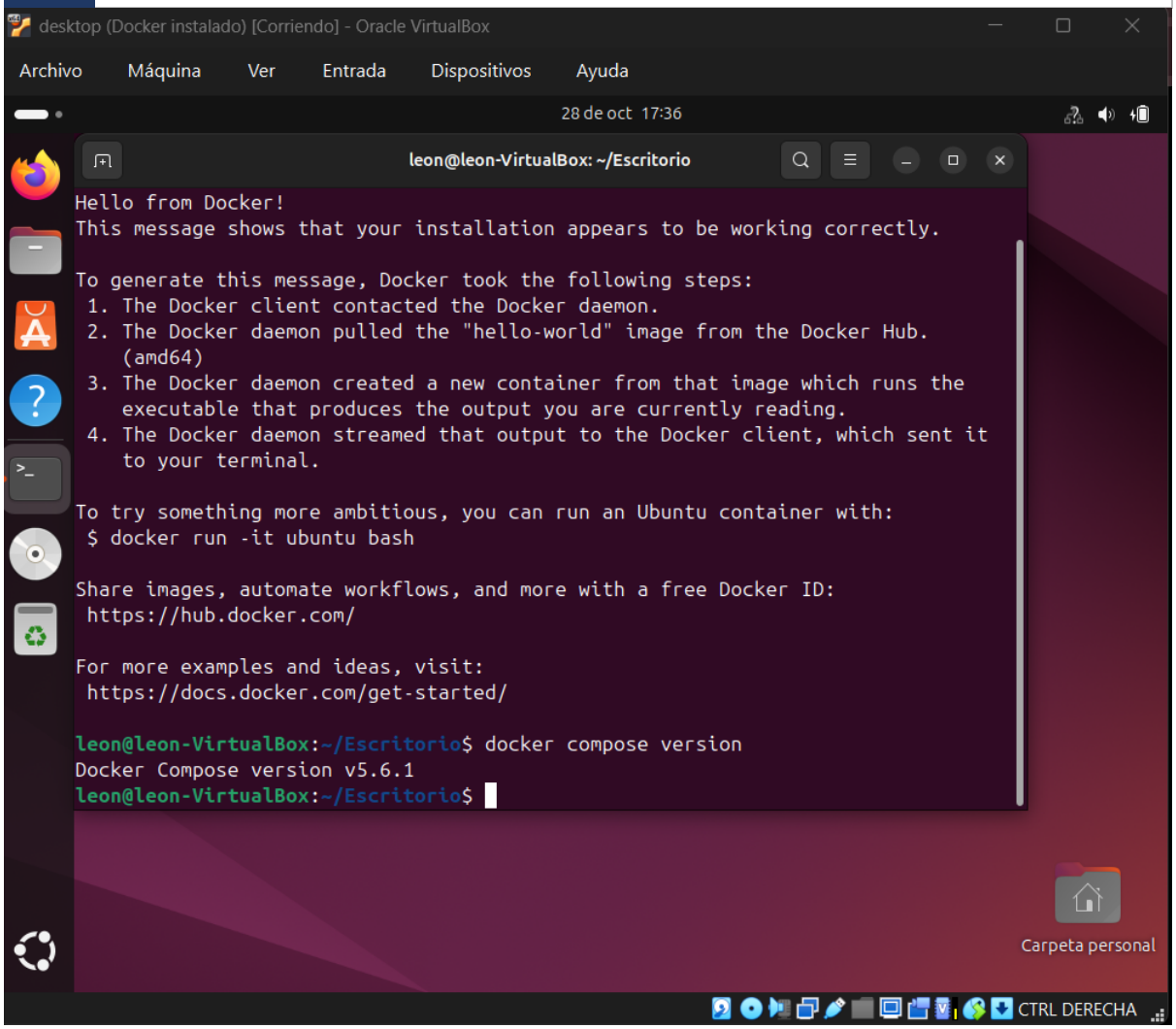
El cambio de grupo no se aplica hasta que vuelves a iniciar sesión. Cierra la sesión de Ubuntu (menú de la esquina superior derecha, Apagar > Cerrar la sesión) y entra de nuevo. Después abre la terminal y comprueba que ya funciona sin sudo:

```
docker run hello-world
docker compose version
```

7

Hacer una instantánea de la máquina (recomendado)

Con la máquina en marcha, en el menú de VirtualBox elige **Máquina > Tomar instantánea** y llámala `Docker instalado`. Una instantánea es una «foto» del estado de la máquina: si más adelante algo se estropea, podrás volver a este punto en segundos sin repetir las fases 1 a 3.



6. Fase 4. Preparar el proyecto

En esta fase vas a crear las carpetas del proyecto desde la terminal y a aprender a usar nano, un editor de texto que funciona dentro de la propia terminal.

1

Crear las carpetas

En la terminal, ejecuta este comando. Crea la carpeta `mi-proyecto` en tu carpeta personal y, dentro de ella, la carpeta `web`. La opción `-p` crea las dos de una vez:

```
mkdir -p ~/mi-proyecto/web
```

2

Entrar en la carpeta del proyecto

El comando `cd` (change directory) sirve para moverse entre carpetas:

```
cd ~/mi-proyecto
```

3

Comprobar dónde estás

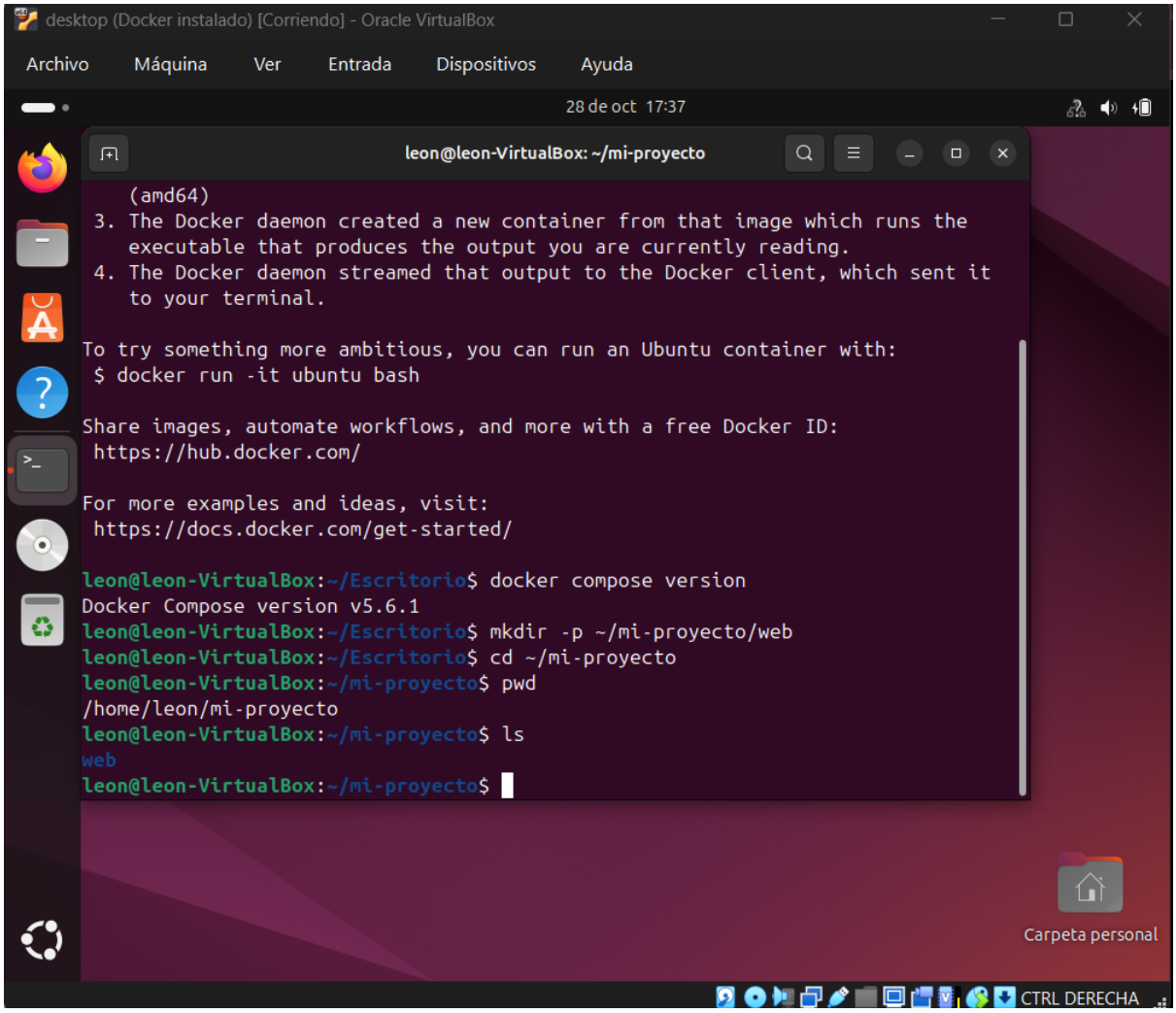
El comando `pwd` muestra la carpeta actual y `ls` lista su contenido. Debe aparecer la carpeta `web`:

```
pwd
ls
```

4

Conocer el editor nano

Para crear y editar archivos usarás `nano`. Se abre escribiendo `nano` seguido del nombre del archivo. Si el archivo no existe, nano lo crea al guardar. Estas son las teclas que necesitas:



Teclas en nano	Qué hacen
`Ctrl + O y luego Intro`	Guardar el archivo
`Ctrl + X`	Salir del editor
`Ctrl + Mayús + V`	Pegar texto copiado (en la terminal de Ubuntu)
`Ctrl + K`	Cortar (borrar) la línea entera donde está el cursor
`Flechas del teclado`	Moverse por el texto

IMPORTANTE: LA TERMINAL SIEMPRE EN MI-PROYECTO

A partir de aquí, todos los comandos de la práctica se ejecutan desde la carpeta `~/mi-proyecto`. Si abres una terminal nueva, lo primero es escribir `cd ~/mi-proyecto`.

Si prefieres un editor con ventanas, puedes usar el «Editor de texto» de Ubuntu: `gnome-text-editor web/index.html`.

7. Fase 5. Crear la página web

Ahora vas a crear la página web que se mostrará dentro del contenedor. No hace falta saber mucho HTML: partirás de una plantilla y la completarás con tu información.

1

Crear el archivo index.html

Desde `~/mi-proyecto`, abre el editor sobre el archivo nuevo. Fíjate en que va dentro de la carpeta `web`:

```
nano web/index.html
```

2

Escribir la plantilla base

Escribe o pega esta estructura mínima:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Mi web</title>
</head>
<body>
  <h1>Escribe aquí tu nombre</h1>
  <p>Escribe aquí una breve presentación.</p>
</body>
</html>
```

3

Completar la página con tu contenido

Modifica y amplía la plantilla para que tu página incluya, como mínimo, los siguientes elementos:

a) Un título principal con tu nombre.

b) Un párrafo de presentación de dos o tres líneas.

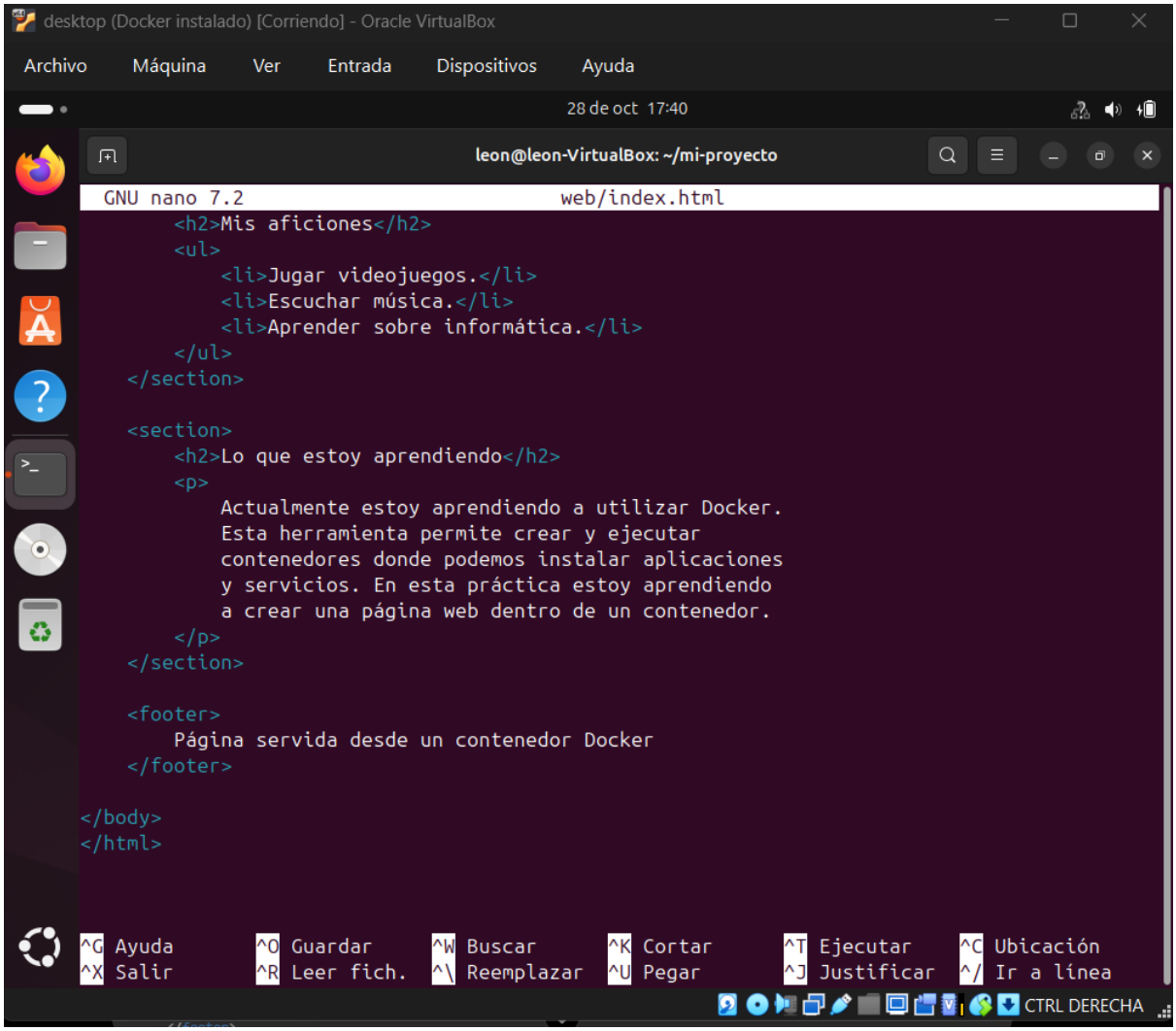
c) Una lista con al menos tres de tus aficiones.

d) Una sección titulada «Lo que estoy aprendiendo» con un párrafo sobre Docker.

e) Un pie de página con el texto «Página servida desde un contenedor Docker».

f) Estilos propios: al menos un color de fondo, un tipo de letra y un color para los títulos.

Cuando termines, guarda con Ctrl + O, Intro, y sal con Ctrl + X.



Etiquetas HTML que vas a necesitar

Etiqueta	Para qué sirve	Ejemplo
`<h1>` y `<h2>`	Títulos principal y secundarios	`<h2>Mis aficiones</h2>`
`<p>`	Párrafo de texto	`<p>Hola, soy...</p>`
`<ul>` y `<li>`	Lista y cada uno de sus elementos	`<ul><li>Leer</li></ul>`
`<section>`	Agrupar una parte de la página	`<section>...</section>`
`<footer>`	Pie de página	`<footer>...</footer>`
`<style>`	Estilos CSS; va dentro de <head>	`body { background: #eef2f7; }`

Propiedades CSS útiles

Propiedad	Qué cambia	Ejemplo
`background`	Color de fondo	`background: #eef2f7;`
`color`	Color del texto	`color: #1f3864;`
`font-family`	Tipo de letra	`font-family: Arial, sans-serif;`

Propiedad	Qué cambia	Ejemplo
`max-width y margin`	Ancho máximo y centrado del contenido	`max-width: 700px; margin: auto;`
`padding`	Espacio interior	`padding: 20px;`
`border-radius`	Esquinas redondeadas	`border-radius: 8px;`

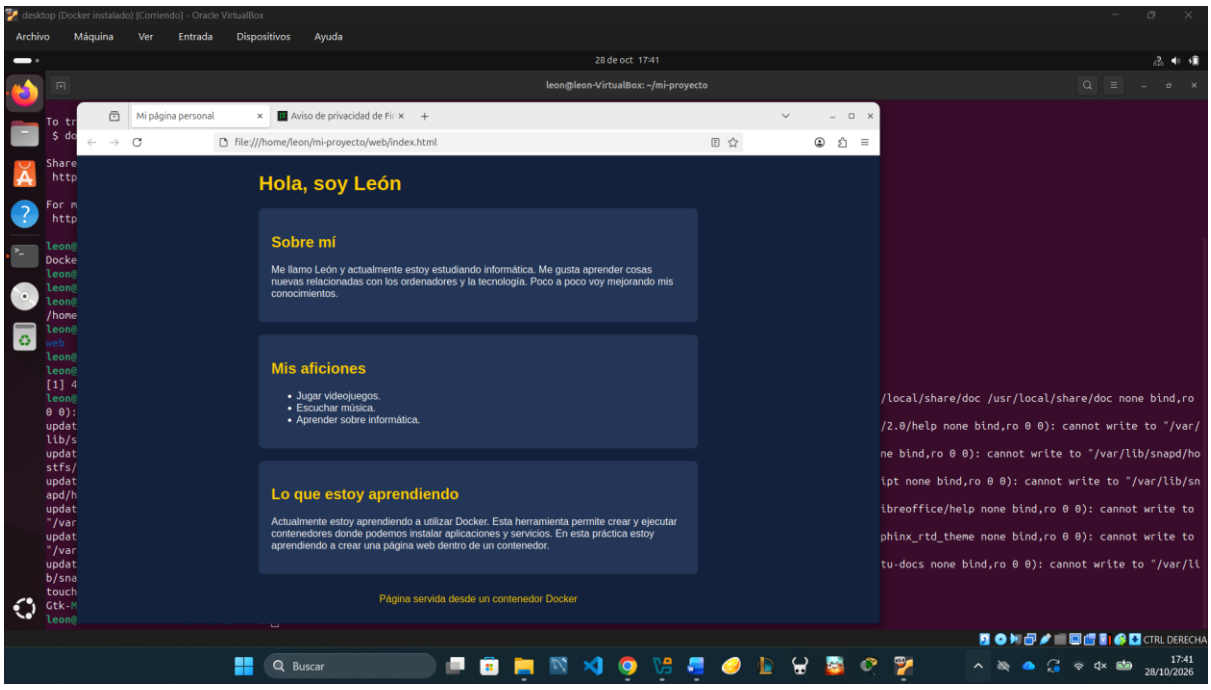
4

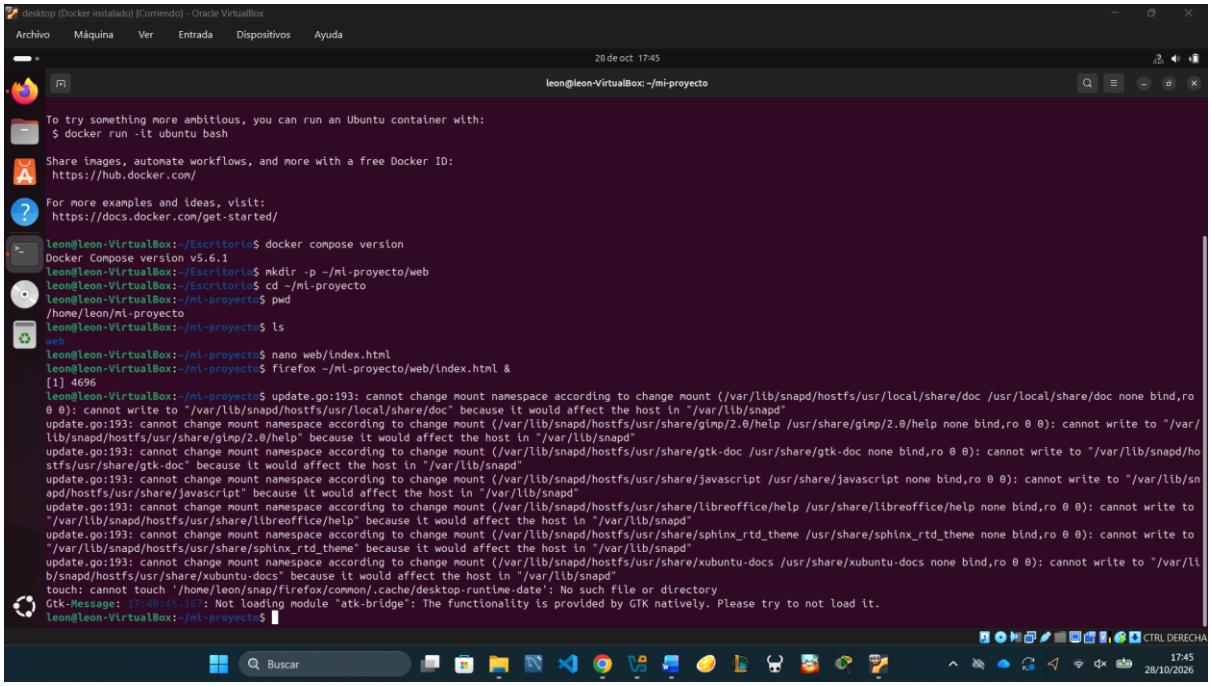
Comprobar la página sin Docker

Abre el archivo directamente en Firefox para revisar que todo se ve bien antes de meterlo en un contenedor:

```
firefox ~/mi-proyecto/web/index.html &
```

El símbolo `&` al final abre Firefox sin bloquear la terminal. Si algo no se ve bien, corrígelo ahora con nano: es mucho más fácil que hacerlo con la web ya dentro de un contenedor.




desktop (Docker instalado) [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
28 de oct 17:45
leon@leon-VirtualBox: ~/mi-proyecto
To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash
Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/
For more examples and ideas, visit:
https://docs.docker.com/get-started/
leon@leon-VirtualBox:~/Escritorio$ docker compose version
Docker Compose version v5.6.1
leon@leon-VirtualBox:~/Escritorio$ mkdir -p ~/mi-proyecto/web
leon@leon-VirtualBox:~/Escritorio$ cd ~/mi-proyecto
leon@leon-VirtualBox:~/mi-proyecto$ pwd
/home/leon/mi-proyecto
leon@leon-VirtualBox:~/mi-proyecto$ ls
web
leon@leon-VirtualBox:~/mi-proyecto$ nano web/index.html
leon@leon-VirtualBox:~/mi-proyecto$ firefox ~/mi-proyecto/web/index.html &
[1] 4696
leon@leon-VirtualBox:~/mi-proyecto$ update.go:193: cannot change mount namespace according to change mount (/var/lib/snapd/hostfs/usr/local/share/doc /usr/local/share/doc none bind,ro 0 0): cannot write to "/var/lib/snapd/hostfs/usr/local/share/doc" because it would affect the host in "/var/lib/snapd"
update.go:193: cannot change mount namespace according to change mount (/var/lib/snapd/hostfs/usr/share/gimp/2.0/help /usr/share/gimp/2.0/help none bind,ro 0 0): cannot write to "/var/lib/snapd/hostfs/usr/share/gimp/2.0/help" because it would affect the host in "/var/lib/snapd"
update.go:193: cannot change mount namespace according to change mount (/var/lib/snapd/hostfs/usr/share/gtk-doc /usr/share/gtk-doc none bind,ro 0 0): cannot write to "/var/lib/snapd/hostfs/usr/share/gtk-doc" because it would affect the host in "/var/lib/snapd"
update.go:193: cannot change mount namespace according to change mount (/var/lib/snapd/hostfs/usr/share/javascript /usr/share/javascript none bind,ro 0 0): cannot write to "/var/lib/snapd/hostfs/usr/share/javascript" because it would affect the host in "/var/lib/snapd"
update.go:193: cannot change mount namespace according to change mount (/var/lib/snapd/hostfs/usr/share/libreoffice/help /usr/share/libreoffice/help none bind,ro 0 0): cannot write to "/var/lib/snapd/hostfs/usr/share/libreoffice/help" because it would affect the host in "/var/lib/snapd"
update.go:193: cannot change mount namespace according to change mount (/var/lib/snapd/hostfs/usr/share/sphinx_rtd_theme /usr/share/sphinx_rtd_theme none bind,ro 0 0): cannot write to "/var/lib/snapd/hostfs/usr/share/sphinx_rtd_theme" because it would affect the host in "/var/lib/snapd"
update.go:193: cannot change mount namespace according to change mount (/var/lib/snapd/hostfs/usr/share/xubuntu-docs /usr/share/xubuntu-docs none bind,ro 0 0): cannot write to "/var/lib/snapd/hostfs/usr/share/xubuntu-docs" because it would affect the host in "/var/lib/snapd"
touch: cannot touch '/home/leon/snap/firefox/common/.cache/desktop-runtime-date': No such file or directory
Gtk-Message: 17:40:45.167: Not loading module "atk-bridge": The functionality is provided by GTK natively. Please try to not load it.
leon@leon-VirtualBox:~/mi-proyecto$
CTRL DERECHA
Buscar
17:45
28/10/2026

8. Fase 6. Empaquetar la web con un Dockerfile

Tu página existe, pero solo como archivo. En esta fase vas a crear una imagen de Docker que contenga un servidor web con tu página dentro. Así, cualquier equipo con Docker podrá ejecutarla exactamente igual.

1

Crear el archivo Dockerfile

Abre nano sobre un archivo llamado exactamente `Dockerfile` dentro de la carpeta `web`. Empieza por D mayúscula y no lleva extensión: Linux distingue entre mayúsculas y minúsculas.

```
nano web/Dockerfile
```

2

Escribir las instrucciones del Dockerfile

El Dockerfile de esta práctica es muy corto. Solo necesita dos instrucciones:

Instrucción FROM: indica la imagen de partida. Usa `nginx:alpine`, que es el servidor web Nginx en una versión muy ligera.

Instrucción COPY: copia archivos al interior de la imagen. Debe copiar todo el contenido de la carpeta actual (`.`) a la carpeta donde Nginx busca las páginas: `/usr/share/nginx/html`.

Escribe tú las dos líneas siguiendo este formato general, guarda y sal:

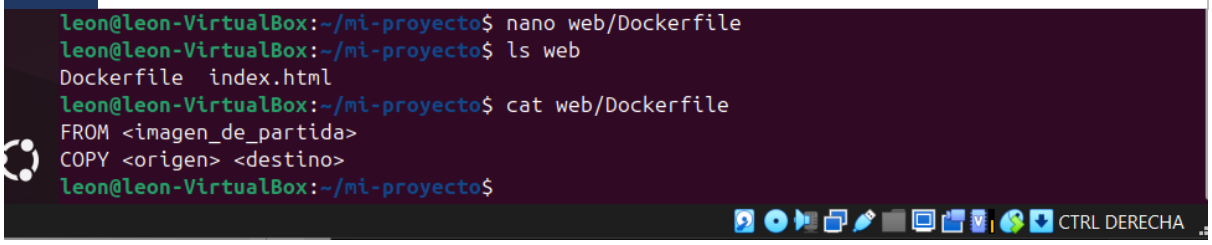
```
FROM <imagen_de_partida>
COPY <origen> <destino>
```

3

Comprobar los archivos

El comando `ls web` debe mostrar los dos archivos. Con `cat` puedes ver el contenido de un archivo sin abrir el editor:

```
ls web
cat web/Dockerfile
```



4

Construir la imagen

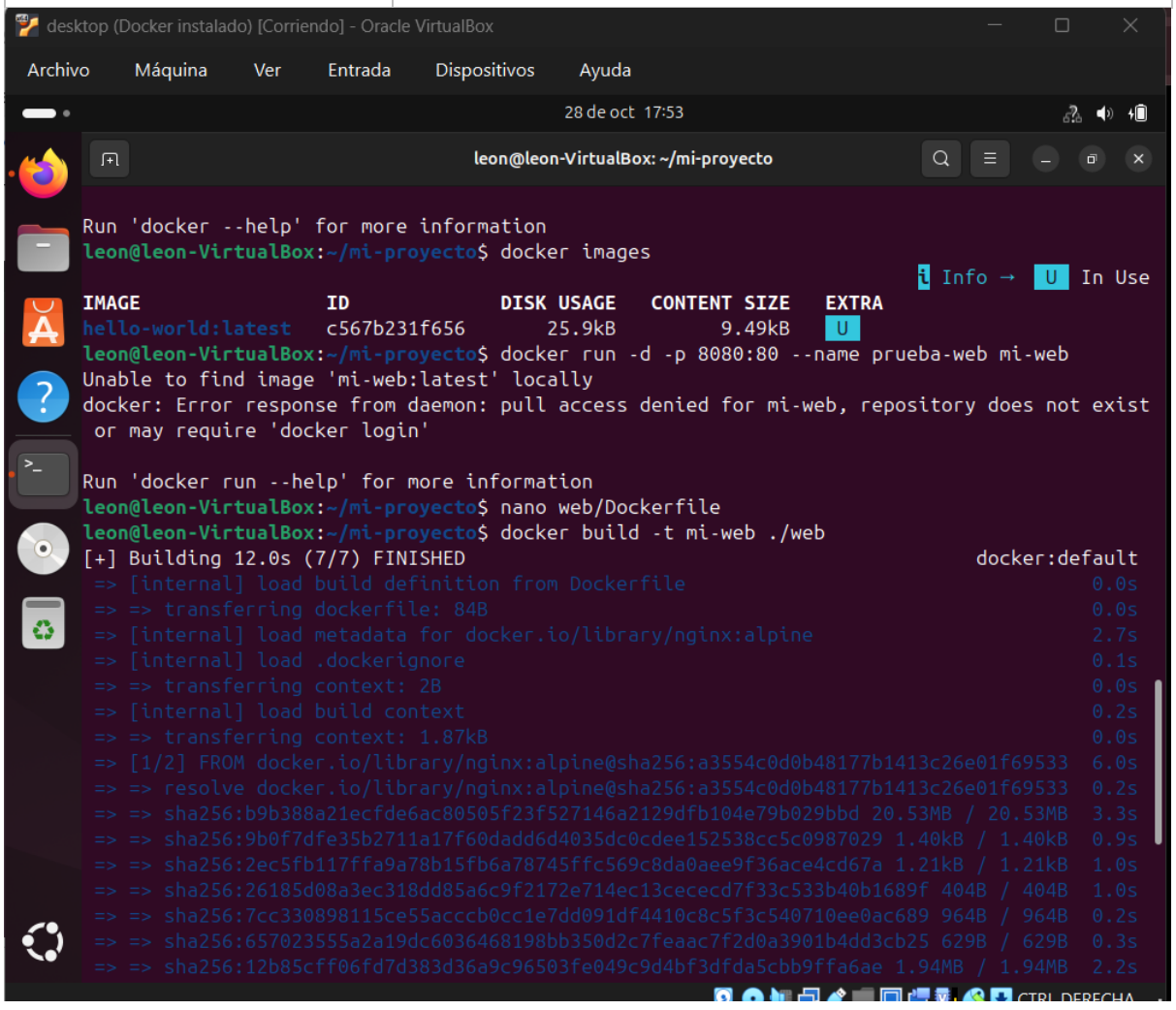
Desde ~/mi-proyecto, ejecuta:

```
docker build -t mi-web ./web
```

```
leon@leon-VirtualBox:~/mi-proyecto$ docker build -t mi-web ./web
[+] Building 0.2s (1/1) FINISHED                                  docker:default
 => [internal] load build definition from Dockerfile                       0.1s
 => => transferring dockerfile: 86B                                        0.0s
 => WARN: InvalidDefaultArgInFrom: Default value for ARG <imagen_de_partida> results   0.1s

 1 warning found (use docker --debug to expand):
 - InvalidDefaultArgInFrom: Default value for ARG <imagen_de_partida> results in empty or in
valid base image name (line 1)
Dockerfile:1
--------------------
   1 | >>> FROM <imagen_de_partida>
   2 |     COPY <origen> <destino>
   3 |
--------------------
ERROR: failed to build: failed to solve: failed to parse stage name "<imagen_de_partida>": i
nvalid reference format
leon@leon-VirtualBox:~/mi-proyecto$
```

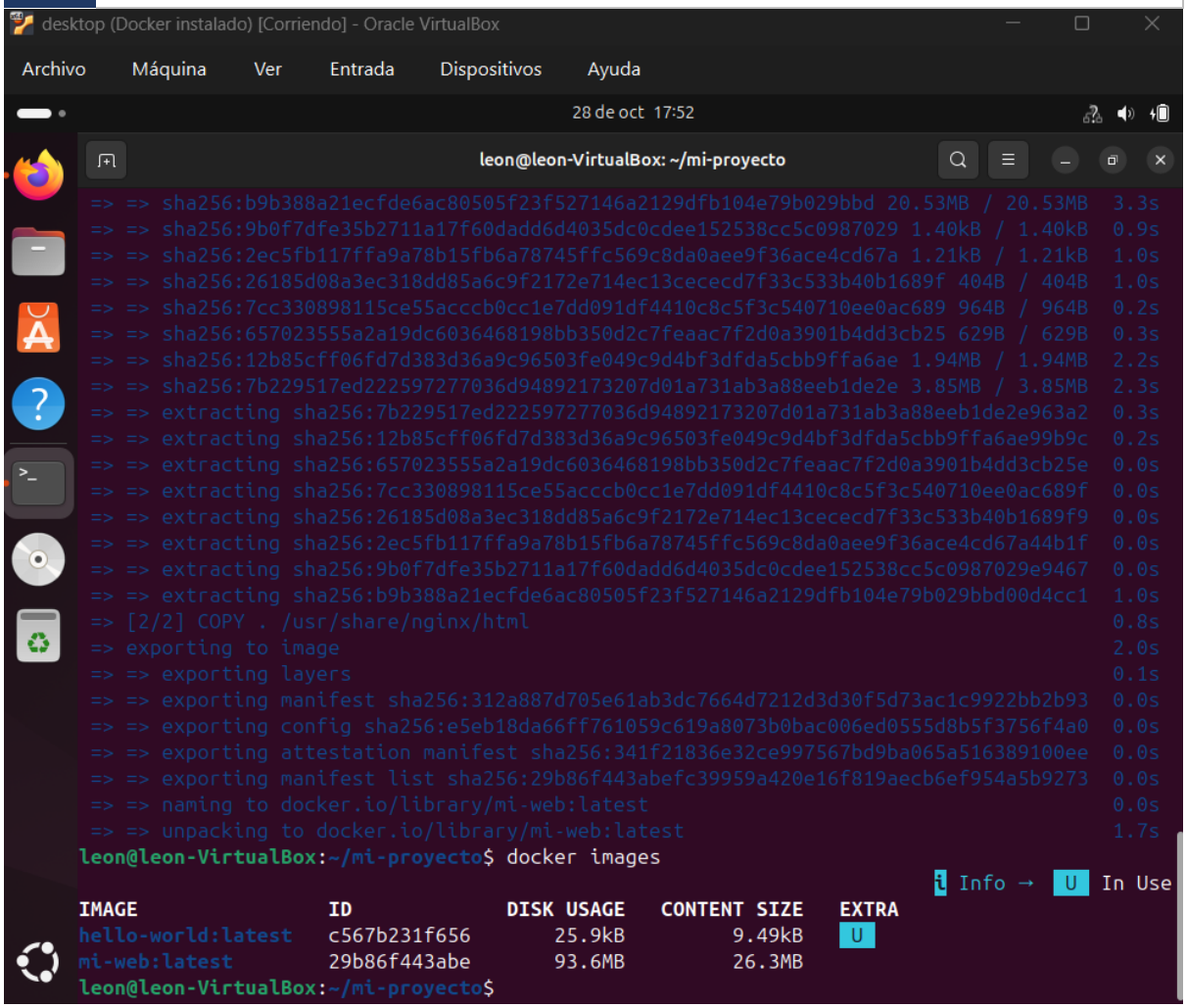
Parte del comando	Significado
`docker build`	Construye una imagen a partir de un Dockerfile
`-t mi-web`	Le pone a la imagen el nombre «mi-web»
`./web`	Carpeta donde está el Dockerfile y los archivos que se copiarán



5 Comprobar que la imagen existe

Lista las imágenes. Entre ellas debe aparecer mi-web:

```
docker images
```



6 Lanzar un contenedor de prueba

Crea un contenedor a partir de tu imagen:

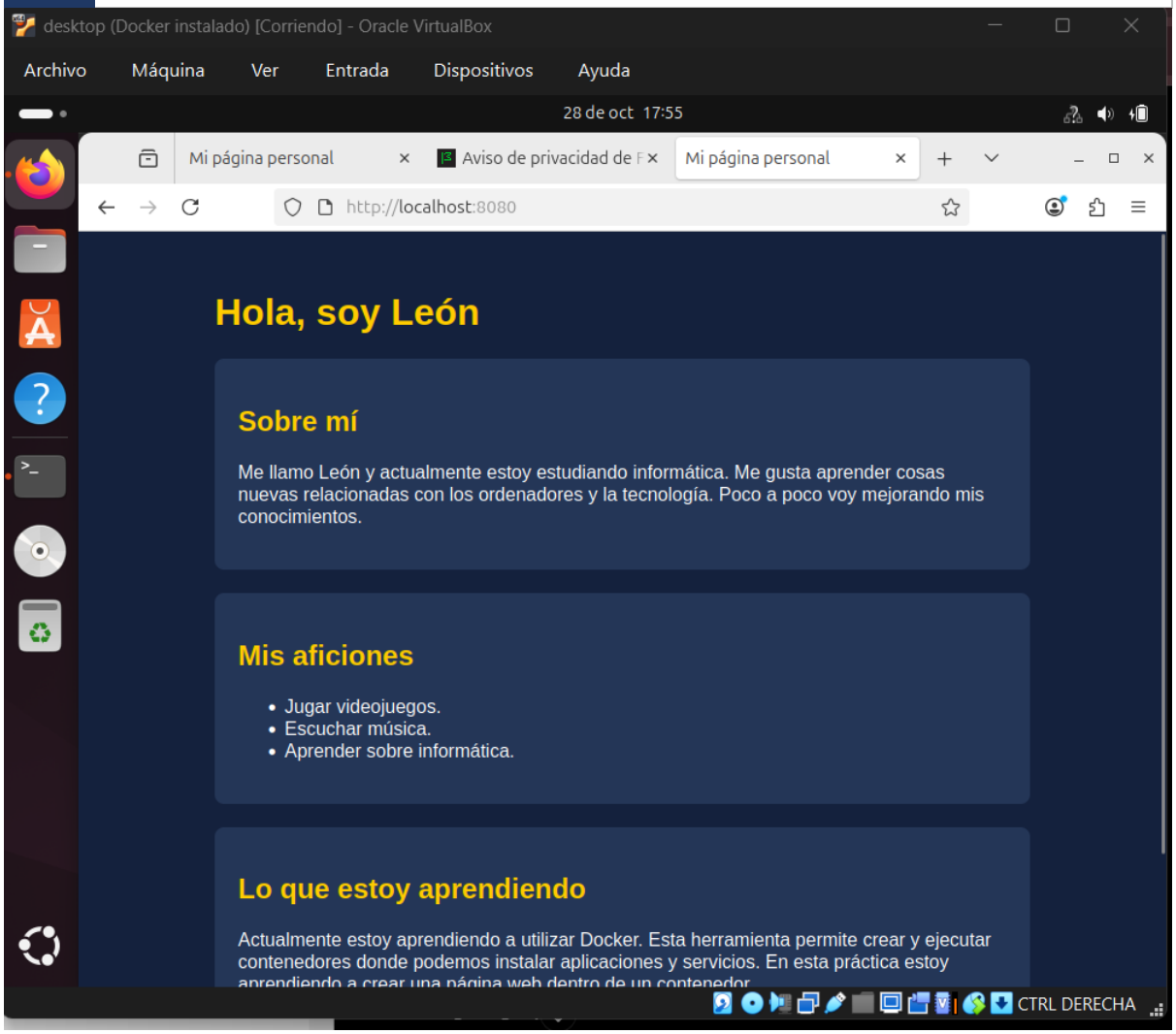
```
docker run -d -p 8080:80 --name prueba-web mi-web
```

```
leon@leon-VirtualBox:~/mi-proyecto$ docker run -d -p 8080:80 --name prueba-web mi-web
e5d03e064ae0c3110f50897132bc7ab38c82961ae98c7e2166b7554531da989c
leon@leon-VirtualBox:~/mi-proyecto$
```

Parte del comando	Significado
`docker run`	Crea y arranca un contenedor
`-d`	Lo ejecuta en segundo plano, sin bloquear la terminal
`-p 8080:80`	Conecta el puerto 8080 de Ubuntu con el 80 del contenedor
`--name prueba-web`	Le pone al contenedor el nombre «prueba-web»
`mi-web`	La imagen a partir de la que se crea el contenedor

7 Ver la web en el navegador

Abre Firefox en Ubuntu y entra en `http://localhost:8080`. Debe aparecer tu página. Ya no la estás abriendo como archivo: te la está sirviendo un servidor web que vive dentro de un contenedor.



8 Detener y eliminar el contenedor de prueba

Este contenedor solo servía para comprobar la imagen. Elimínalo para dejar libre el puerto 8080:

```
docker stop prueba-web
docker rm prueba-web
```

```
leon@leon-VirtualBox:~/mi-proyecto$ docker stop prueba-web
prueba-web
leon@leon-VirtualBox:~/mi-proyecto$ docker rm prueba-web
prueba-web
leon@leon-VirtualBox:~/mi-proyecto$
```

> **NO TE SALTES EL PASO 8**
>
> Si el contenedor de prueba sigue en marcha, en la fase siguiente aparecerá un error indicando que el puerto 8080 ya está ocupado («port is already allocated»).

9. Fase 7. Unirlo todo con Docker Compose

Hasta ahora has lanzado un contenedor con un comando largo. Con dos contenedores, cada uno con sus opciones, eso se vuelve incómodo. Docker Compose permite describirlos todos en un único archivo y arrancarlos juntos con una sola orden.

1

Crear el archivo compose.yaml

Desde `~/mi-proyecto` (no dentro de `web`), abre nano sobre el archivo nuevo:

```
nano compose.yaml
```

2

Conocer las reglas del formato YAML

Los archivos YAML organizan la información mediante sangrías, como un esquema. Hay tres reglas que debes respetar:

Sangría con espacios: usa siempre dos espacios por nivel. Nunca uses el tabulador.

Dos puntos: cada propiedad se escribe como nombre, dos puntos, un espacio y el valor.

Guiones: los elementos de una lista empiezan por un guion y un espacio.

3

Escribir el archivo compose.yaml

Escribe el archivo de forma que describa lo siguiente. Debajo tienes una estructura de ayuda con los huecos que debes completar.

Servicio «web»: se construye a partir de la carpeta `./web` (propiedad `build`) y publica el puerto `8080` hacia el `80` del contenedor.

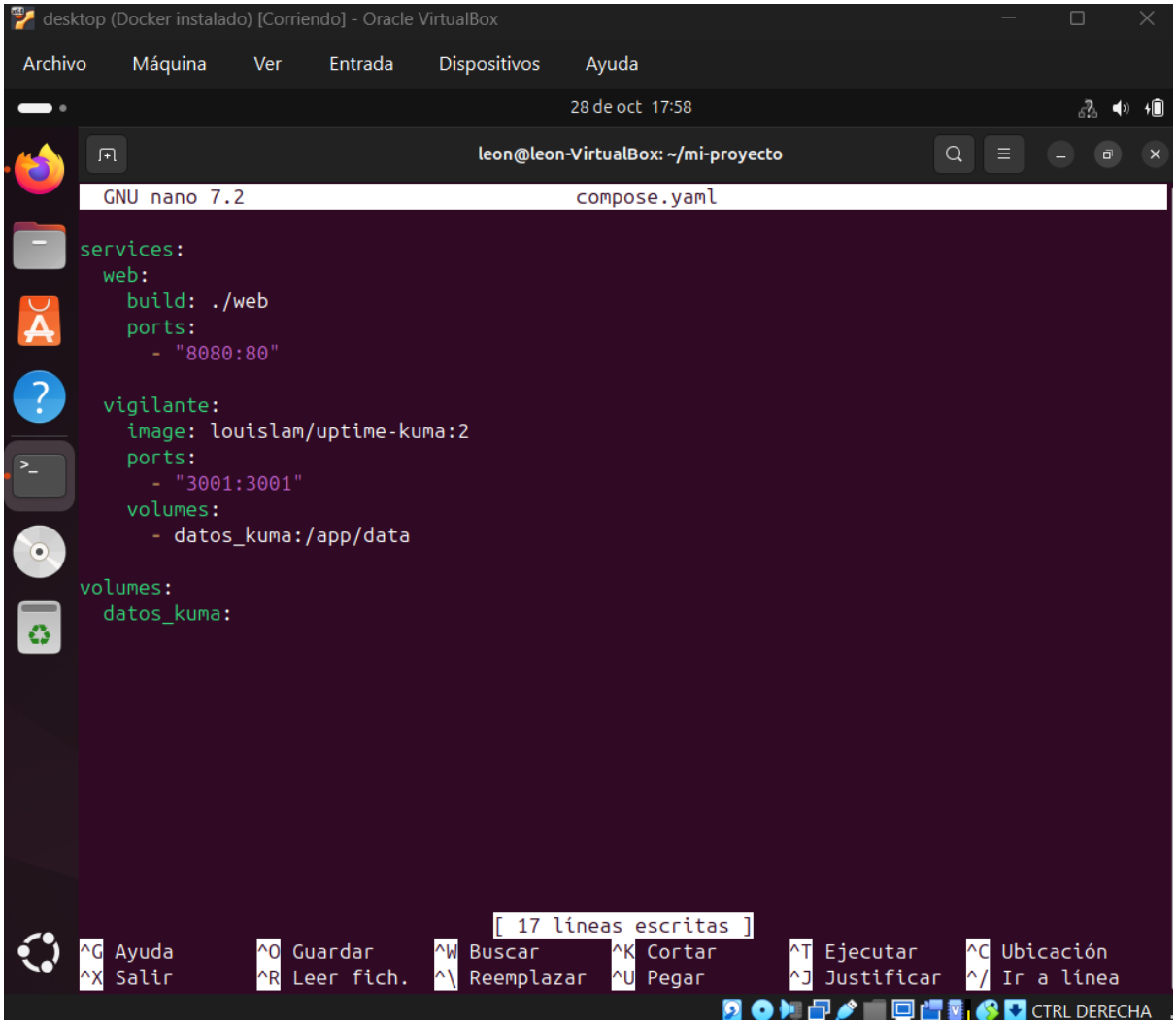
Servicio «vigilante»: usa la imagen `louislam/uptime-kuma:2`, publica el puerto `3001` hacia el `3001` y monta el volumen `datos_kuma` en la ruta `/app/data`.

Volúmenes: al final del archivo se declara el volumen `datos_kuma`.

```
services:
  web:
    build: ________
    ports:
      - "____:__"

  vigilante:
    image: ________________
    ports:
      - "____:____"
    volumes:
      - __________:________

volumes:
  __________:
```



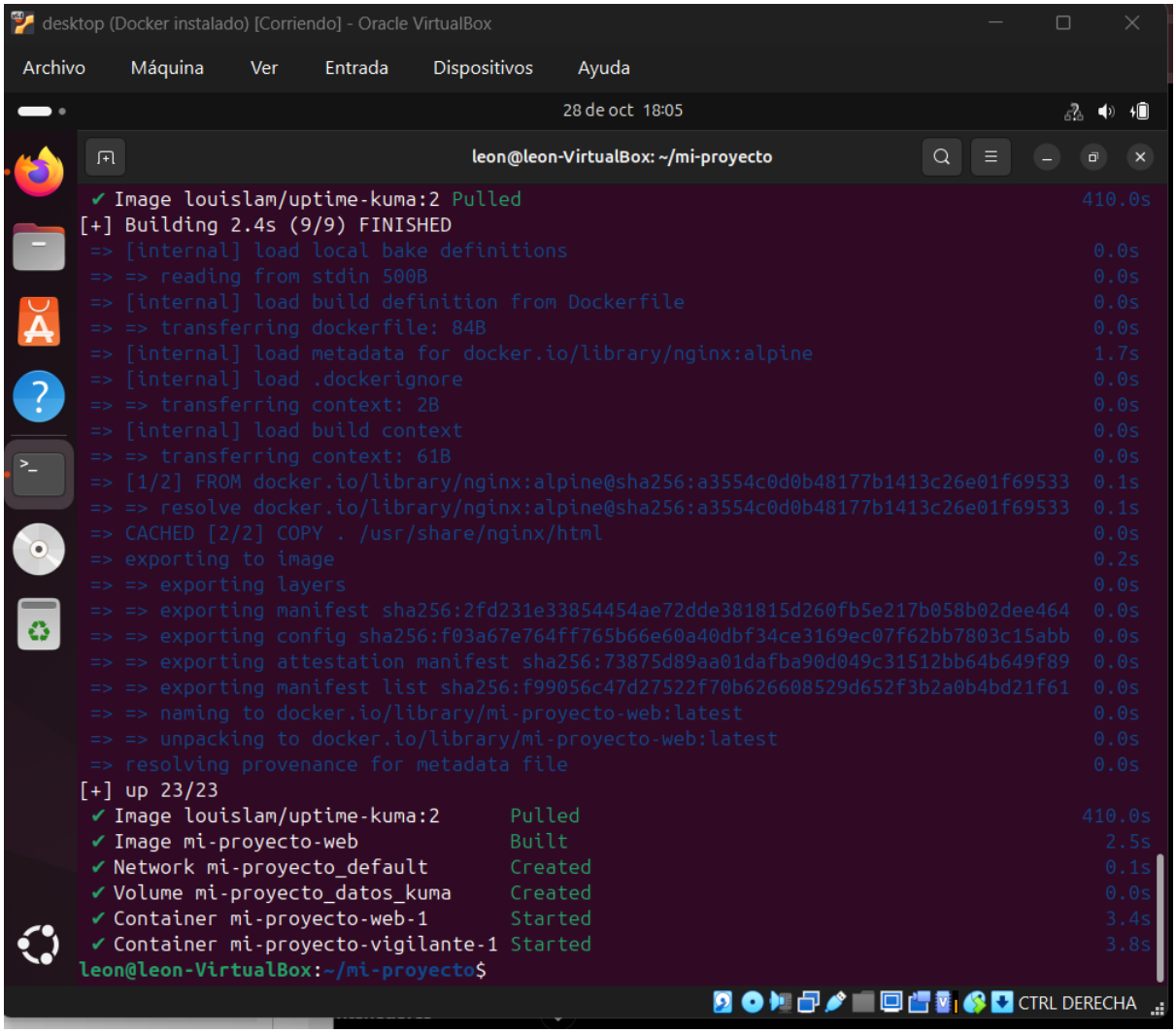
4 **Arrancar los dos contenedores**

Guarda, sal de nano y ejecuta:

```
docker compose up -d --build
```

Parte del comando	Significado
`docker compose up`	Crea y arranca todos los servicios del archivo compose.yaml
`-d`	En segundo plano
`--build`	Construye (o reconstruye) la imagen de la web antes de arrancar

La primera vez tardará un poco, porque tiene que descargar la imagen de Uptime Kuma.

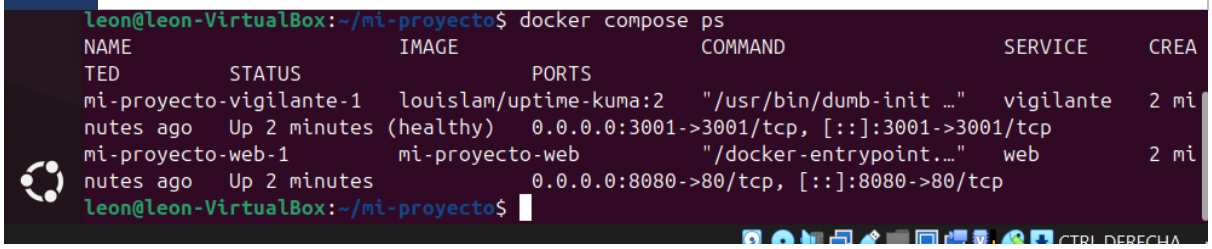


5

Comprobar el estado de los contenedores

Deben aparecer dos contenedores, uno por servicio, ambos en estado «Up» (en marcha):

```
docker compose ps
```



6

Comprobar la web

Entra de nuevo en `http://localhost:8080` desde Firefox. Debe verse tu página, ahora servida por el contenedor que ha creado Docker Compose.

> **SI ALGO FALLA**
>
> Para ver qué está pasando dentro de un servicio, consulta sus registros con `docker compose logs web` o `docker compose logs vigilante`. La mayoría de errores en esta fase se deben a sangrías incorrectas en el archivo YAML.

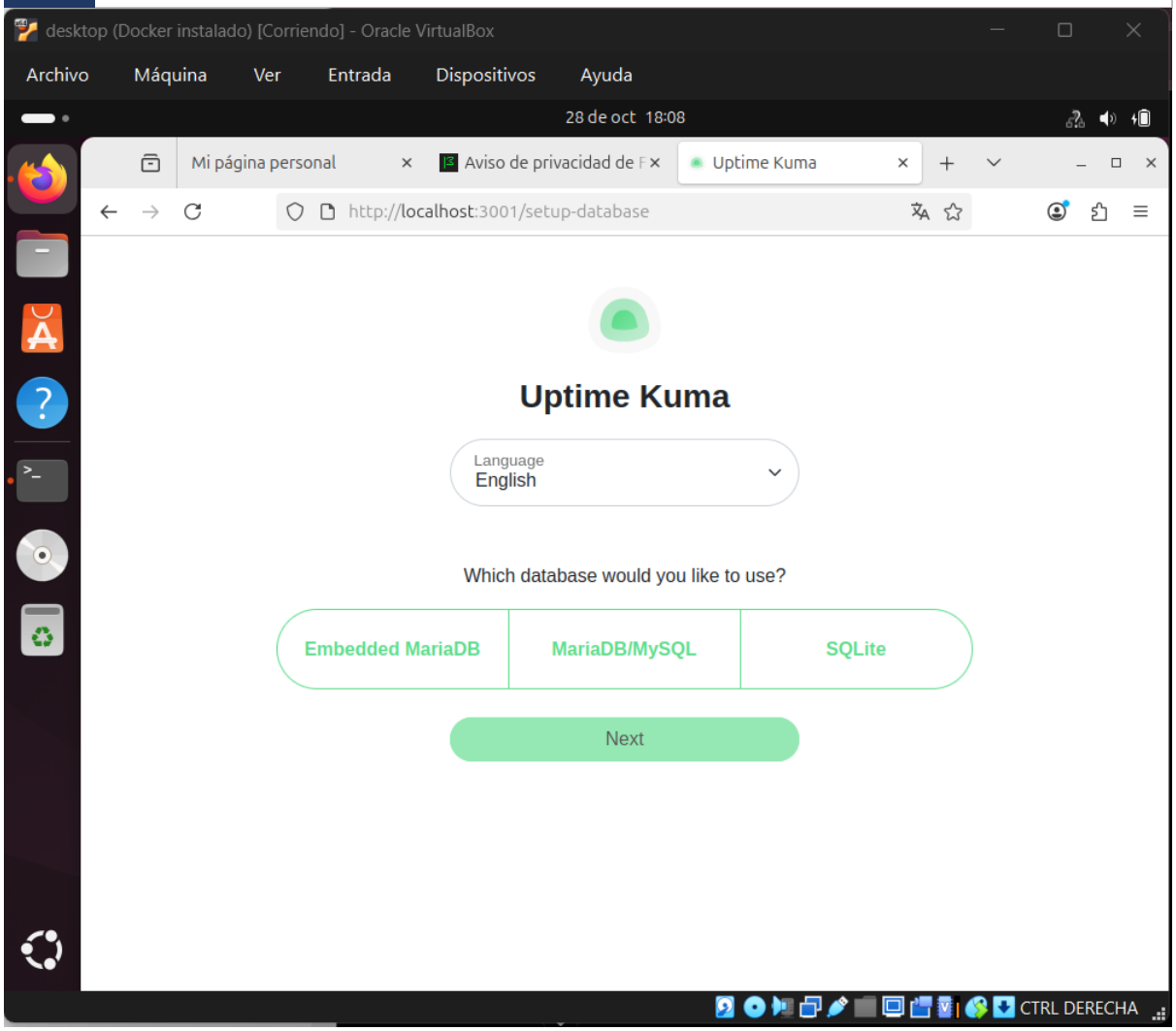
10. Fase 8. Configurar el vigilante

El contenedor de Uptime Kuma ya está en marcha, pero todavía no vigila nada. En esta fase vas a crear tu cuenta de administrador y a darle de alta tu web.

1

Abrir el panel

En Firefox, entra en `http://localhost:3001`. La primera vez, Uptime Kuma puede tardar entre medio minuto y un minuto en estar listo; si la página no carga, espera un poco y recárgala.



2

Elegir la base de datos

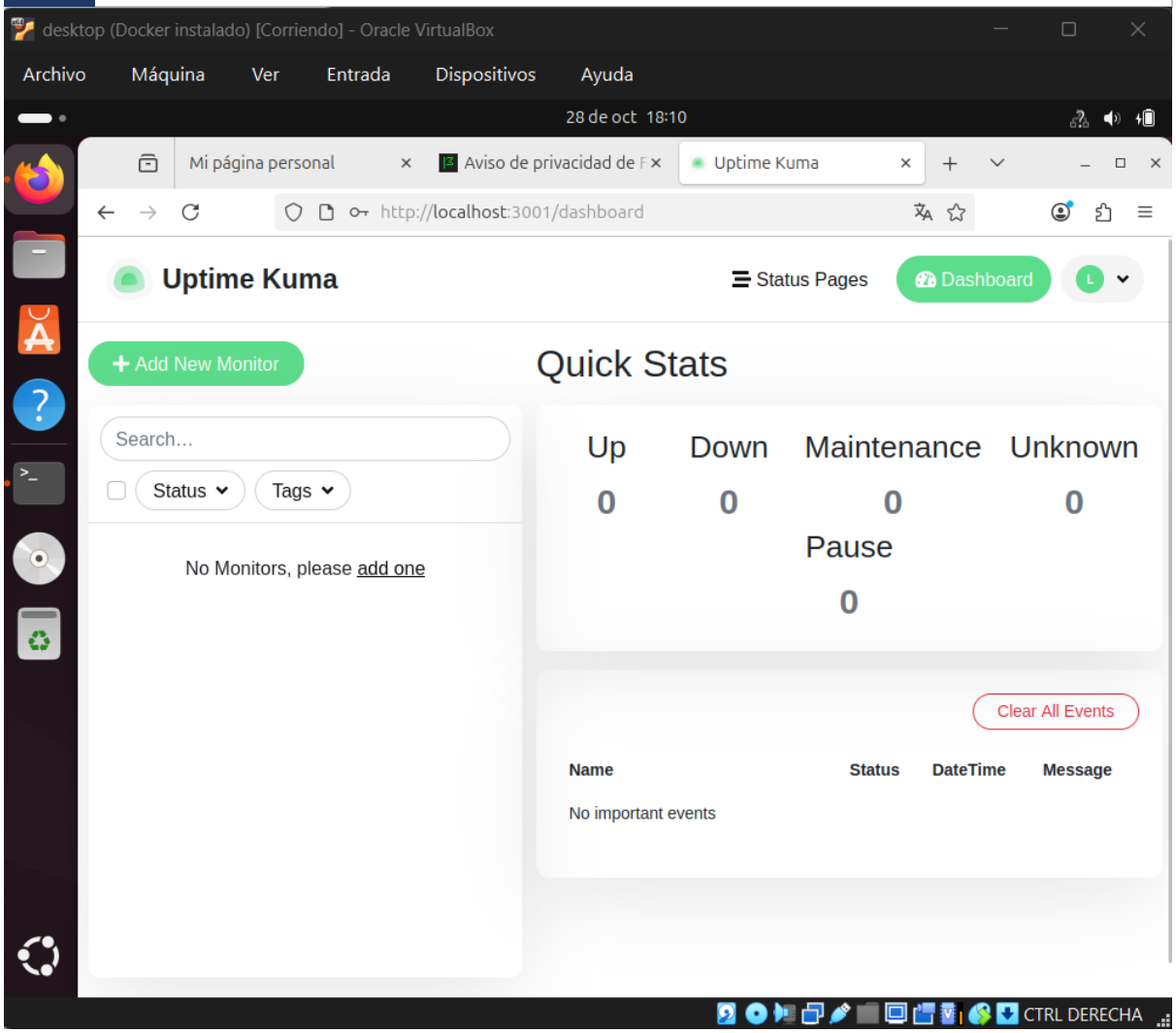
Es posible que lo primero que aparezca sea una pantalla preguntando qué base de datos usar. Elige **SQLite**, que es la opción más sencilla: guarda todo en un archivo dentro del volumen.

Si esta pantalla no aparece, continúa con el paso siguiente.

3

Crear la cuenta de administrador

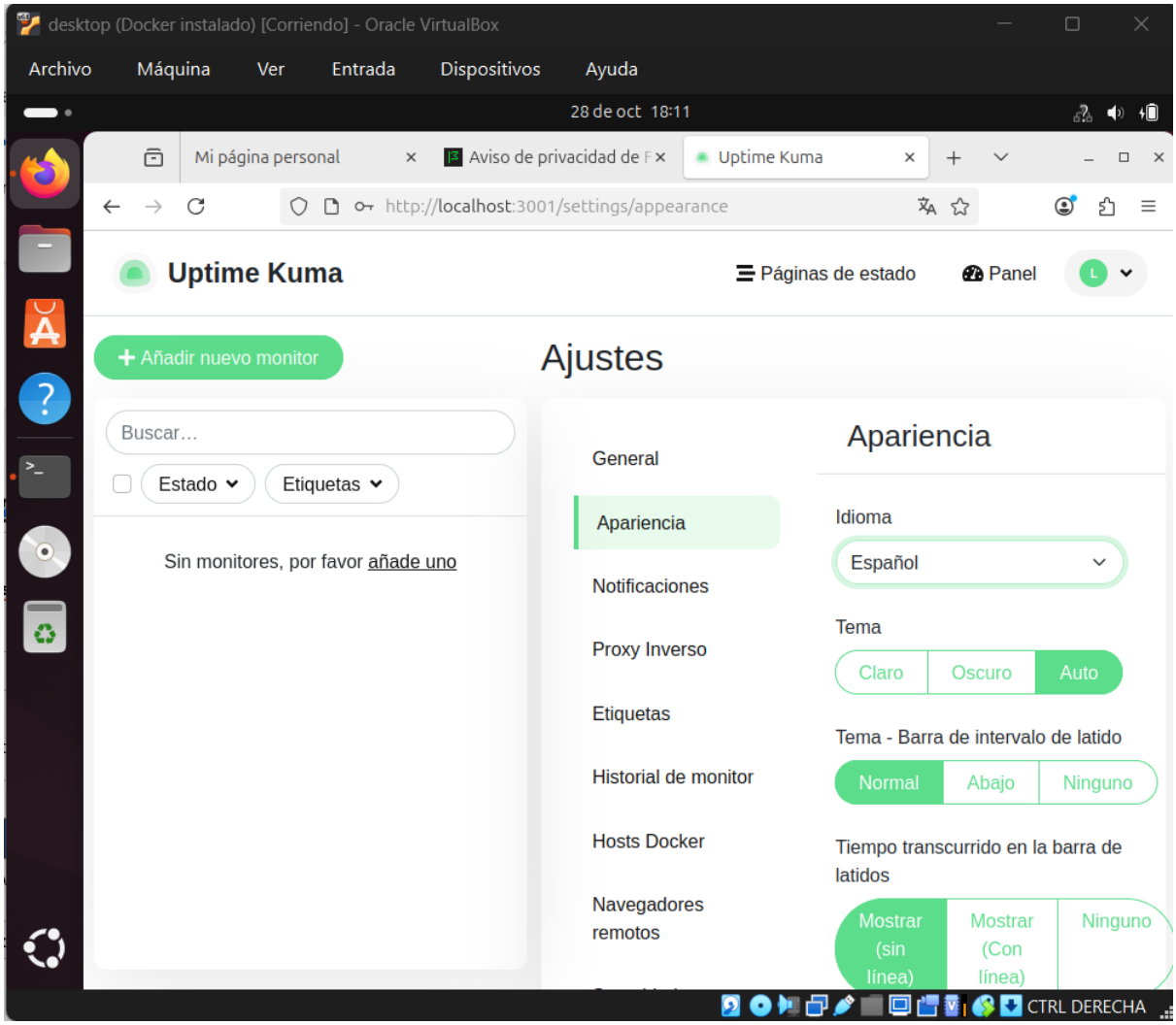
Escribe un nombre de usuario y una contraseña (dos veces) y pulsa el botón de crear. Anota los datos: los necesitarás más adelante.



4

Poner el panel en español (si aparece en inglés)

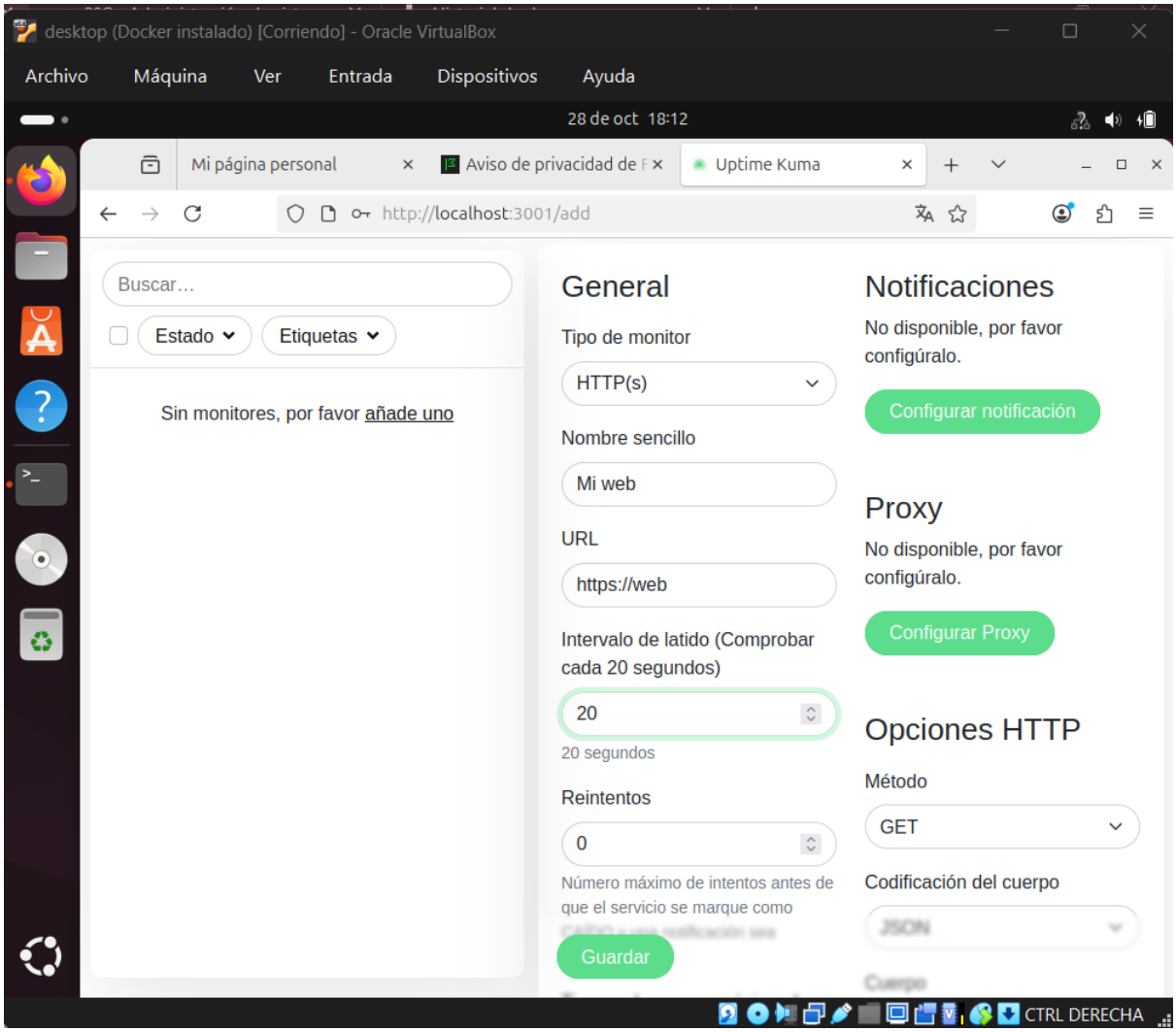
Haz clic en tu perfil (arriba a la derecha) y entra en **Settings > Appearance**. En **Language** elige **Español**.



5 Añadir un monitor para tu web

Pulsa **Añadir nuevo monitor** (Add New Monitor) y rellena los campos así:

Campo	Valor que debes escribir	Explicación
Tipo de monitor	HTTP(s)	Comprobará la web haciendo peticiones como un navegador
Nombre amigable	Mi web	Nombre con el que aparecerá en el panel
URL	http://web	Dirección de la web dentro de la red interna
Intervalo de latido	20	Comprobará la web cada 20 segundos (el mínimo permitido)
Reintentos	0	Marcará la web como caída al primer fallo, sin esperar

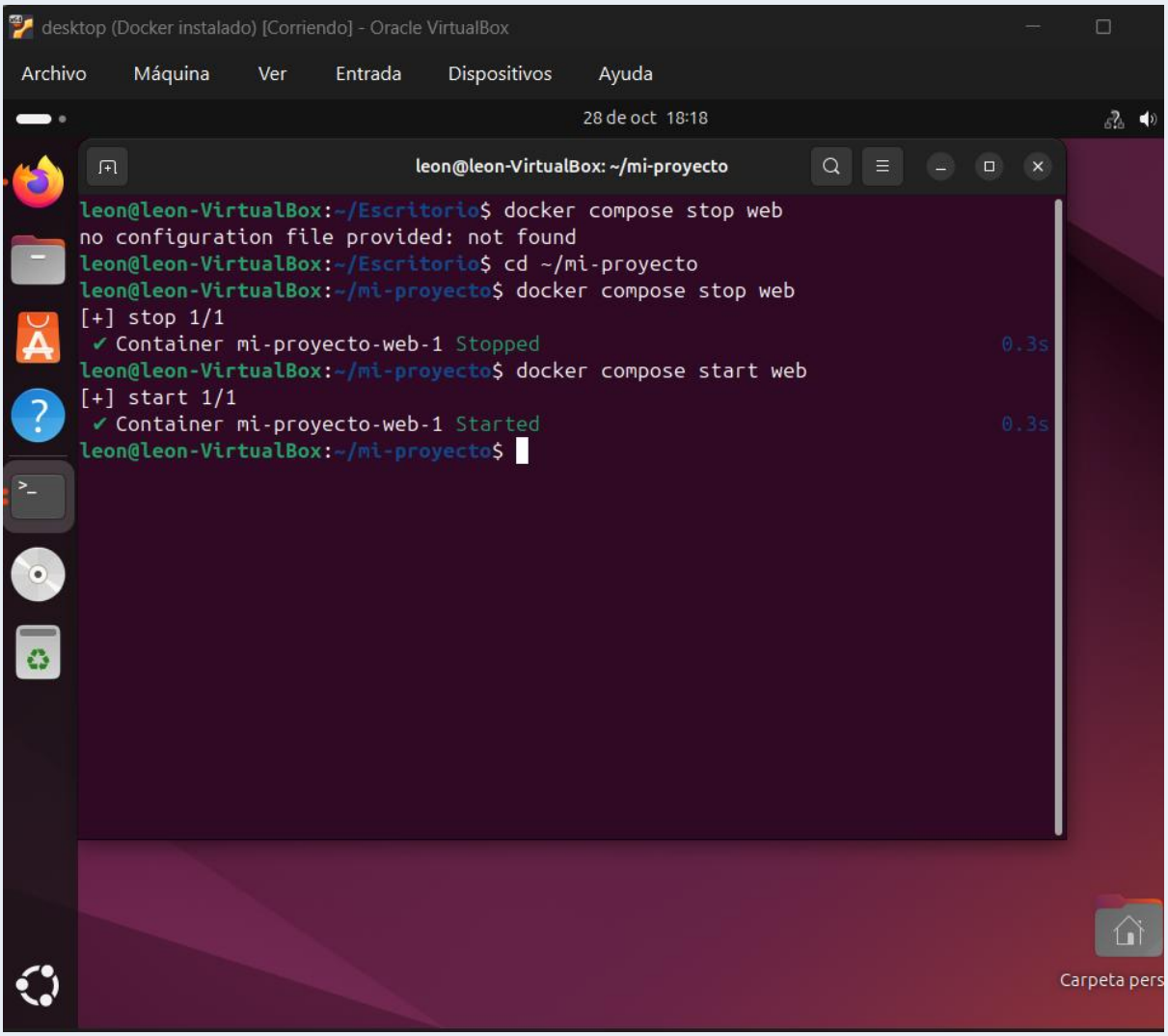


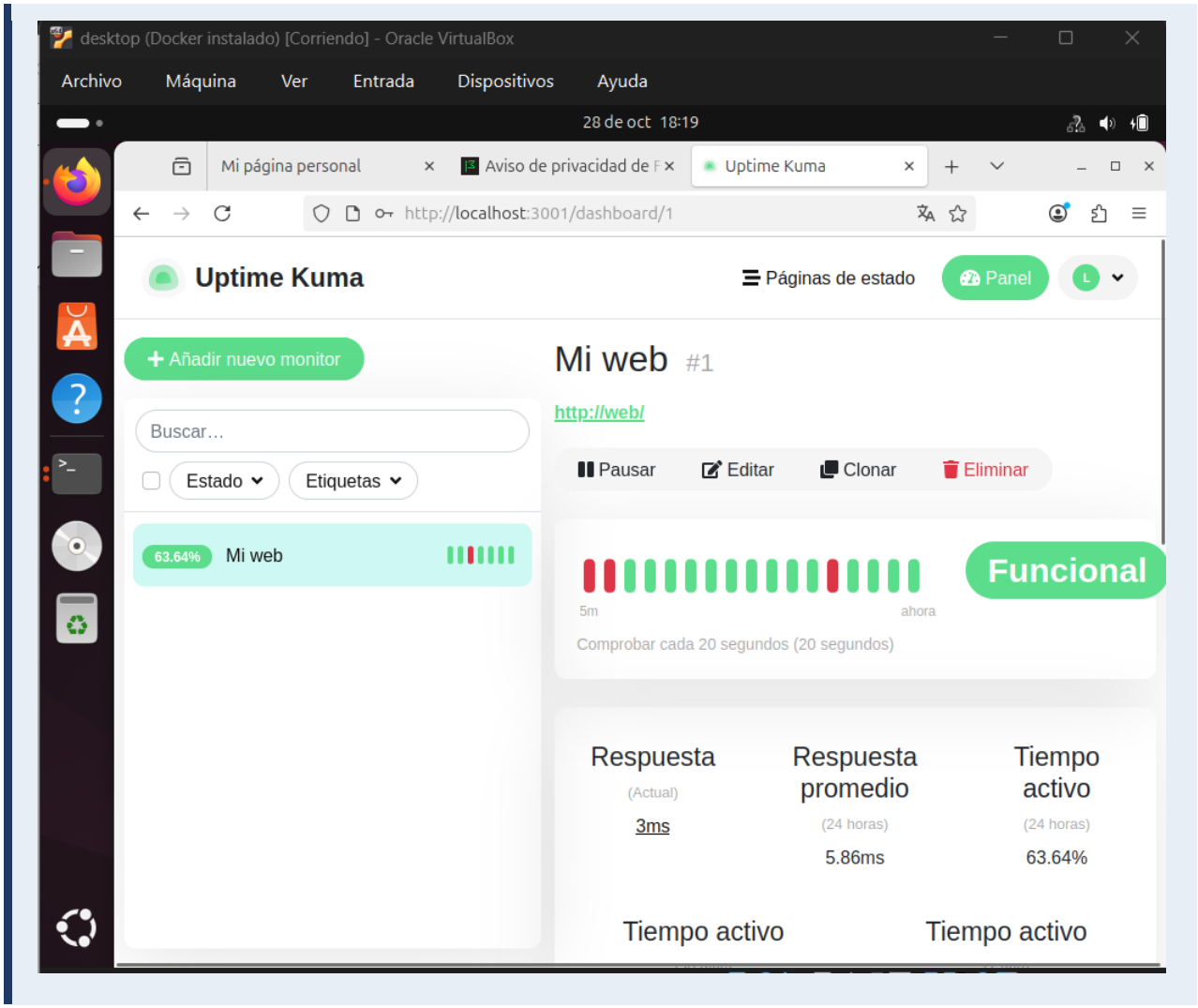
6 **Guardar el monitor**

Pulsa Guardar. En pocos segundos el monitor aparecerá en verde (Up) y empezará a dibujar la gráfica de tiempo de respuesta.

ATENCIÓN A LA URL

Escribe `http://web` y no `http://localhost:8080`. El vigilante vive dentro de su propio contenedor: para él, «localhost» es él mismo, no Ubuntu. Dentro de la red de Docker Compose, cada contenedor encuentra a los demás por el nombre de su servicio, que en este caso es «web».




desktop (Docker instalado) [Corriendo] - Oracle VirtualBox
Archivo
Máquina
Ver
Entrada
Dispositivos
Ayuda
28 de oct 18:19
Mi página personal
Aviso de privacidad de F
Uptime Kuma
http://localhost:3001/dashboard/1
Uptime Kuma
Páginas de estado
Panel
Añadir nuevo monitor
Buscar…
Estado
Etiquetas
63.64%
Mi web
Mi web #1
http://web/
Pausar
Editar
Clonar
Eliminar
Funcional
5m
ahora
Comprobar cada 20 segundos (20 segundos)
Respuesta
(Actual)
3ms
Respuesta promedio
(24 horas)
5.86ms
Tiempo activo
(24 horas)
63.64%
Tiempo activo
Tiempo activo

11. Fase 9. La prueba de fuego: tirar la web

Vas a simular que tu web se cae, como le ocurre a cualquier servicio real, y vas a comprobar que el vigilante lo detecta. Coloca Firefox con el panel a un lado de la pantalla de Ubuntu y la terminal al otro, para ver ambos a la vez.

1

Detener el contenedor de la web

Detén solo el servicio web. El vigilante sigue funcionando:

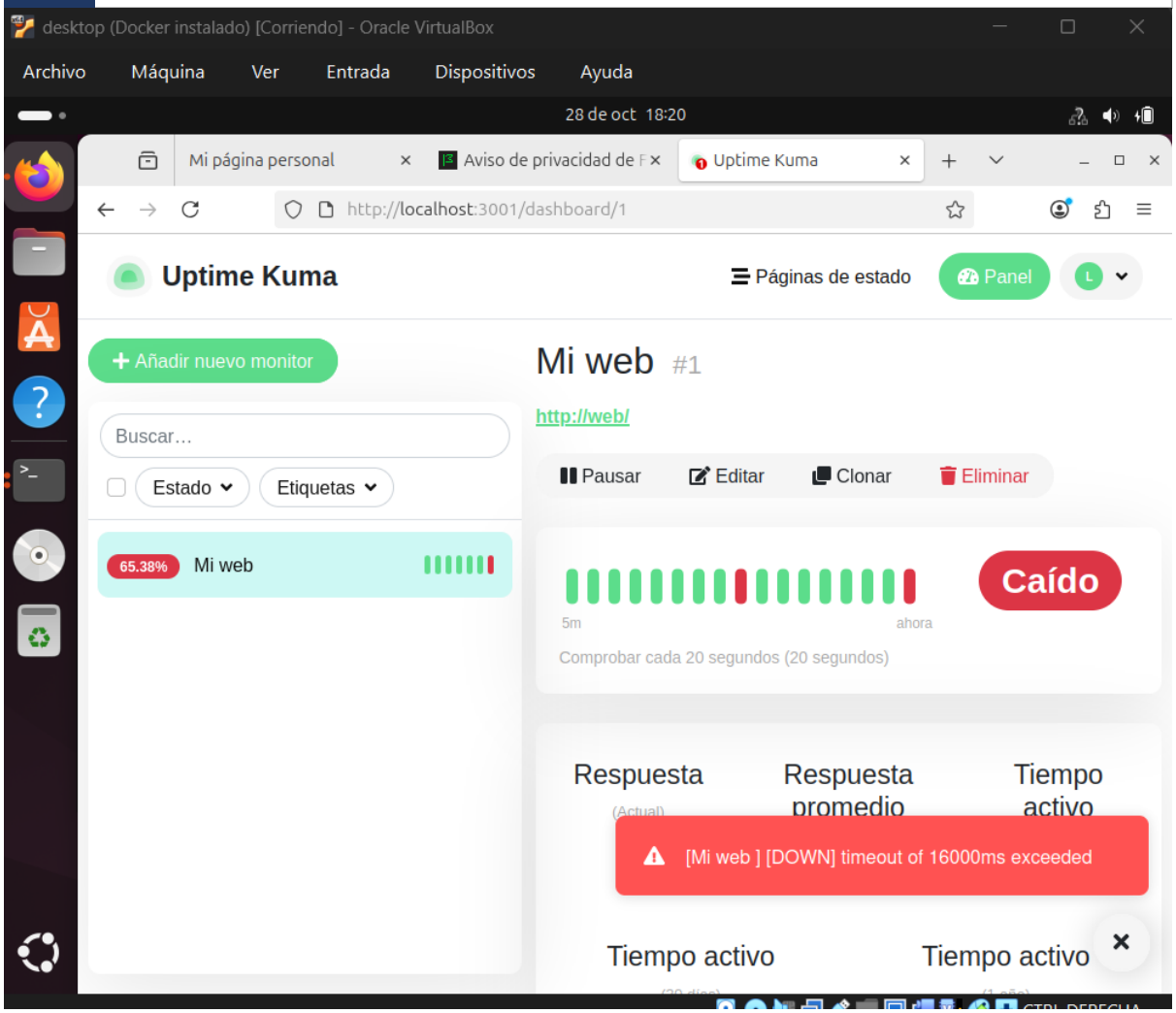
```
docker compose stop web
```

2

Observar el panel

Espera hasta 20 segundos. El monitor «Mi web» pasará a rojo con el estado de caída (Down) y mostrará el mensaje de error recibido.

Comprueba también que `http://localhost:8080` ya no carga.

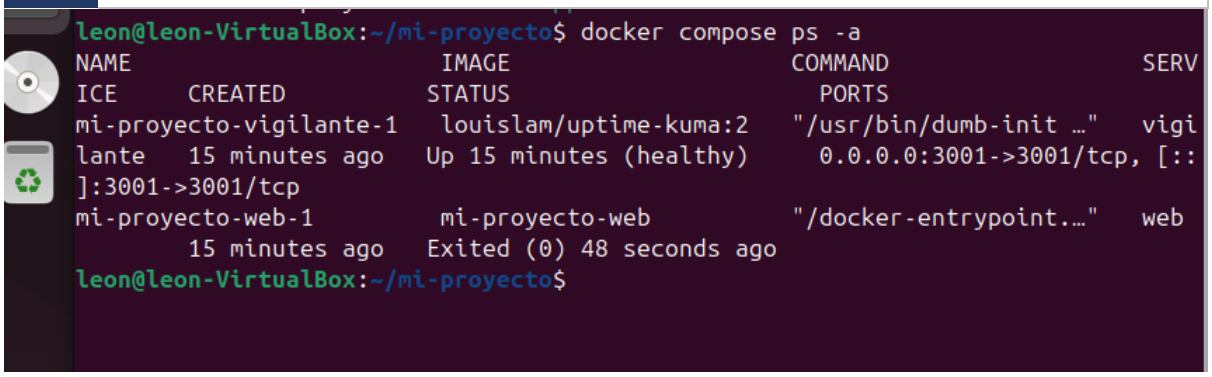


3

Ver el estado desde la terminal

Comprueba que el contenedor de la web está detenido y el del vigilante sigue en marcha:

```
docker compose ps -a
```



4

Recuperar la web

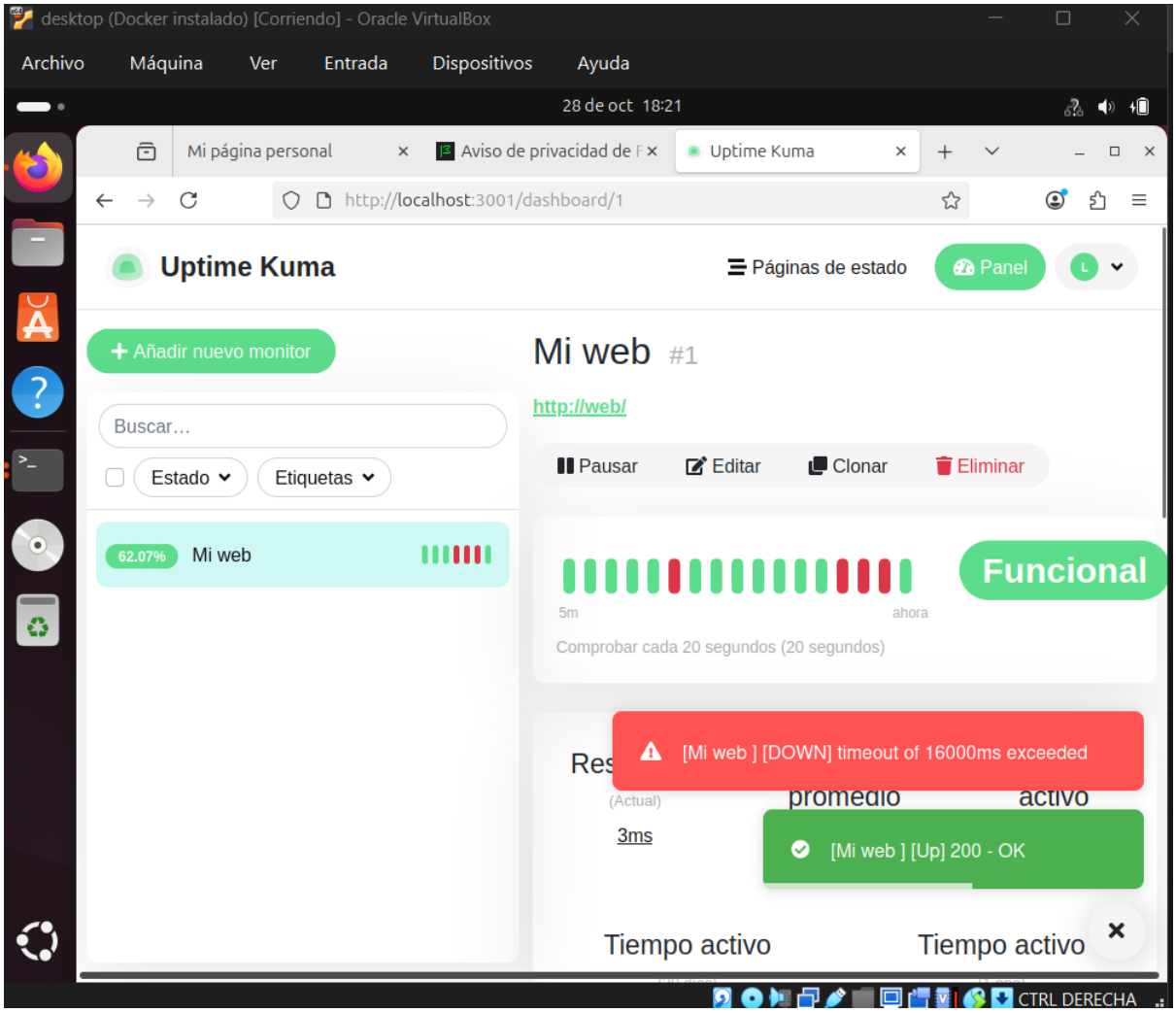
Vuelve a arrancar el servicio:

```
docker compose start web
```

5

Observar la recuperación

En menos de 20 segundos el monitor volverá a verde. En la barra de historial verás el tramo rojo que corresponde al tiempo que la web estuvo caída.



LO QUE ACABAS DE COMPROBAR

Has simulado una caída real y has visto cómo un sistema de monitorización la detecta y registra. Además, has comprobado que los contenedores son independientes: detener uno no afecta al otro.

12. Fase 10. Página de estado, cambios y persistencia

Parte A: crear una página de estado pública

1

Crear la página

En el panel, entra en **Páginas de estado** (Status Pages) y pulsa **Nueva página de estado**. Como nombre escribe `Estado de mis servicios` y como identificador (Slug) `estado`. Pulsa Siguiente.

2

Añadir el monitor y guardar

En la página de edición, añade el monitor «Mi web» con el desplegable y pulsa Guardar.

La página quedará disponible en `http://localhost:3001/status/estado`.

Parte B: actualizar la web

3

Modificar la página

Abre de nuevo `nano web/index.html`, añade una frase nueva (por ejemplo, una cuarta afición), guarda y recarga `http://localhost:8080`.

El cambio no aparece. Es correcto: la imagen contiene una copia del archivo hecha al construirla, no el archivo original.

4

Reconstruir y relanzar

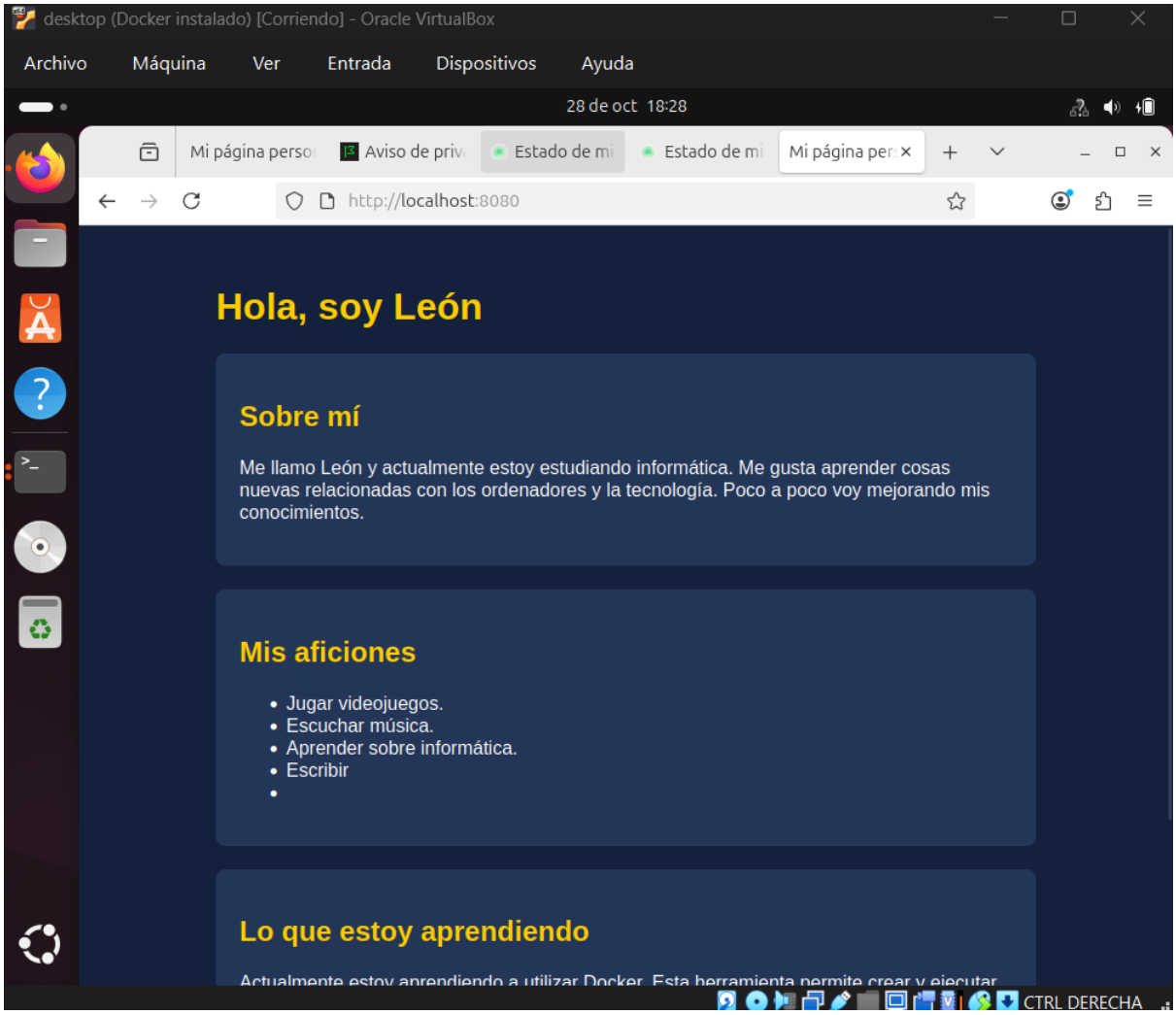
Para que el cambio se vea, hay que reconstruir la imagen y recrear el contenedor:

```
docker compose up -d --build
```

5

Comprobar el cambio

Recarga `http://localhost:8080`. Ahora sí aparece la nueva frase.



Parte C: comprobar la persistencia

6

Eliminar los contenedores

Este comando detiene y **elimina** los contenedores y la red, pero conserva los volúmenes:

```
docker compose down
```

7

Volver a crearlos

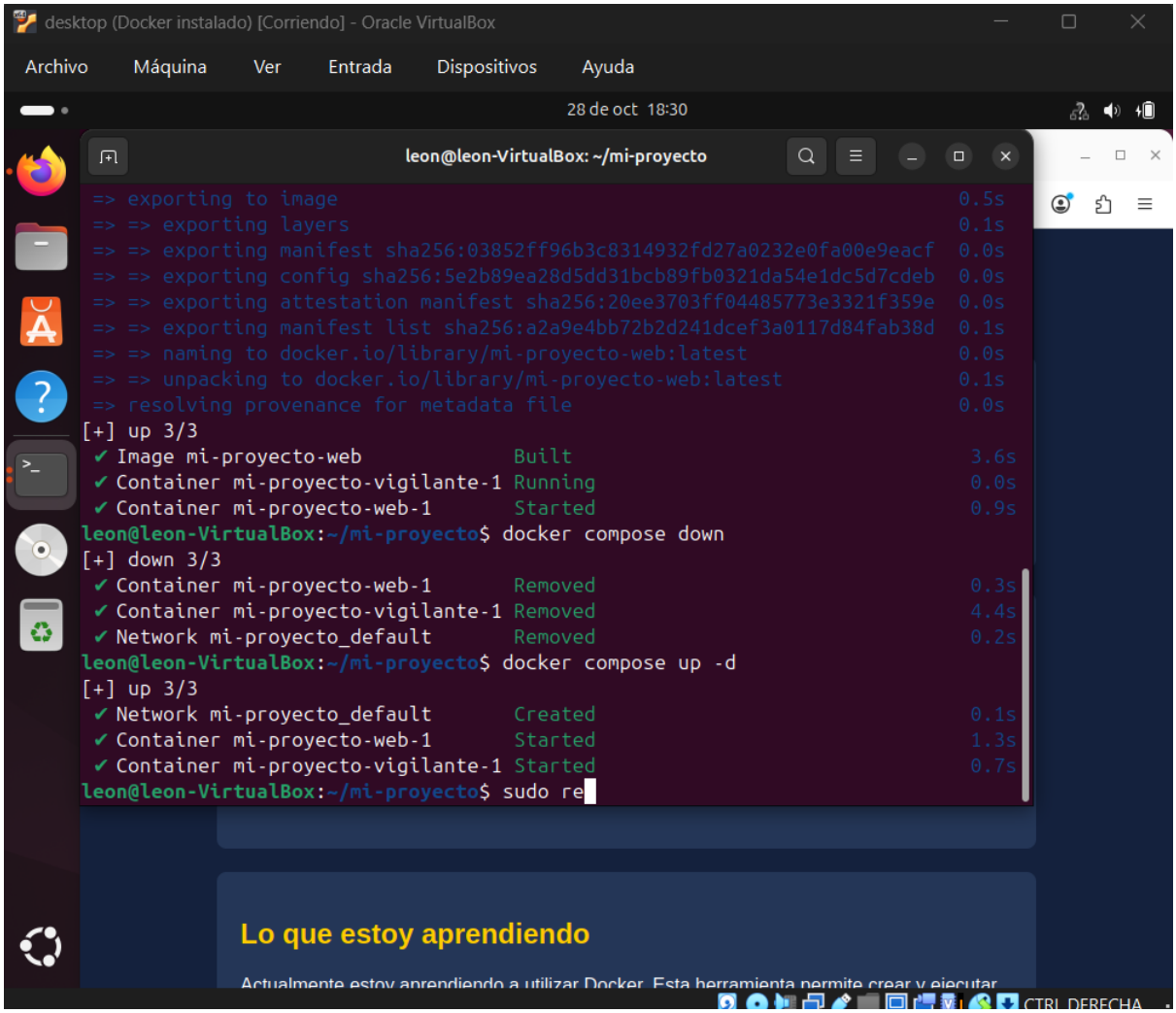
Arranca de nuevo el proyecto:

```
docker compose up -d
```

8

Comprobar que los datos siguen ahí

Entra en `http://localhost:3001`. Aunque el contenedor es nuevo, tu usuario, el monitor y la página de estado siguen existiendo, porque estaban guardados en el volumen `datos_kuma`.



9

Comprobar que Docker sobrevive a un reinicio

Reinicia la máquina virtual completa. Al volver a entrar, abre la terminal y comprueba el estado:

```
sudo reboot
# Tras iniciar sesión de nuevo:
cd ~/mi-proyecto
docker compose ps -a
```

Verás que los contenedores están detenidos: Docker arranca con el sistema, pero los contenedores no, salvo que se indique lo contrario. Arráncalos con `docker compose up -d`.

13. Fase 11. Acceder desde el equipo anfitrión

Hasta ahora has usado Firefox dentro de Ubuntu. En esta fase vas a abrir la web y el panel desde el navegador de tu propio ordenador. Para ello hay que «abrir un camino» a través de la máquina virtual.

Por qué no funciona directamente

Por defecto, VirtualBox conecta la máquina virtual a Internet mediante NAT: la máquina puede salir a Internet, pero desde tu ordenador no se puede entrar en ella. Es como un piso con puerta de salida pero sin timbre. El reenvío de puertos instala ese timbre: lo que llegue a un puerto de tu ordenador se redirige a un puerto de la máquina virtual.

Tramo	Puerto	Quién lo conecta
Tu ordenador → máquina virtual	8080 → 8080 y 3001 → 3001	Reenvío de puertos de VirtualBox
Máquina virtual → contenedor	8080 → 80 y 3001 → 3001	Docker (sección ports de compose.yaml)

1 **Abrir la configuración de red**

Con la máquina encendida o apagada, en VirtualBox selecciona `Ubuntu-Docker` y entra en **Configuración > Red > Adaptador 1**. Comprueba que está conectado a **NAT**.

2 **Abrir el reenvío de puertos**

Despliega **Avanzadas** y pulsa el botón **Reenvío de puertos**.

3 **Añadir las dos reglas**

Pulsa el botón de añadir regla (el signo + a la derecha) dos veces y rellena las reglas así. Deja en blanco las columnas de IP:

Nombre	Protocolo	Puerto anfitrión	Puerto invitado
web	TCP	8080	8080
vigilante	TCP	3001	3001

4 **Guardar**

Pulsa Aceptar en la ventana de reglas y otra vez Aceptar en la configuración.

5 **Probar desde tu ordenador**

Asegúrate de que los contenedores están en marcha en Ubuntu. Después, en el navegador de tu ordenador (no en el de Ubuntu), entra en `http://localhost:8080` y en `http://localhost:3001`. Verás tu web y el panel del vigilante.

SI NO FUNCIONA

Comprueba que ningún programa de tu ordenador usa ya los puertos 8080 o 3001. Si es así, usa otros puertos en la columna «Puerto anfitrión» (por ejemplo, 18080 y 13001) y accede con ellos.

14. Fase 12. Limpieza y entrega

Limpieza final

Antes de limpiar, asegúrate de haber hecho todas las capturas de pantalla que se piden en el apartado de entrega.

1

Eliminar contenedores, red y volumen

Añadiendo `-v` se eliminan también los volúmenes. Esto borra definitivamente la configuración del vigilante:

```
docker compose down -v
```

2

Eliminar las imágenes (opcional)

Para liberar espacio en la máquina virtual:

```
docker rmi mi-web mi-proyecto-web louislam/uptime-kuma:2 nginx:alpine hello-world
```

3

Comprimir el proyecto para entregarlo

Desde tu carpeta personal, crea un archivo comprimido con el proyecto. Para pasarlo a tu ordenador puedes usar la carpeta compartida de VirtualBox, el portapapeles o subirlo desde Firefox a la plataforma de entrega.

```
cd ~
tar -czf mi-proyecto.tar.gz mi-proyecto
```

Qué debes entregar

- El archivo `mi-proyecto.tar.gz` con los archivos `compose.yaml`, `web/Dockerfile` y `web/index.html`.
- Una carpeta llamada capturas con las siguientes capturas de pantalla, numeradas:

Captura	Qué debe mostrar
01	La configuración de la máquina virtual en VirtualBox (memoria, procesadores y disco)
02	La salida de docker run hello-world sin sudo
03	Tu página web en Firefox en http://localhost:8080
04	La salida de docker compose ps con los dos contenedores en marcha
05	El panel de Uptime Kuma con el monitor «Mi web» en verde
06	El monitor «Mi web» en rojo después de detener la web
07	El monitor de nuevo en verde, con el tramo rojo visible en el historial
08	La página de estado en http://localhost:3001/status/estado
09	La web con el cambio realizado en la Parte B de la fase 10

Captura	Qué debe mostrar
10	Las reglas de reenvío de puertos en VirtualBox
11	La web o el panel abiertos desde el navegador de tu ordenador

Criterios de evaluación

Criterio	Puntos
Máquina virtual creada con la configuración pedida y Ubuntu instalado	1
Docker instalado desde el repositorio oficial y usable sin sudo	1
La página web cumple los requisitos de contenido y estilo	1,5
El Dockerfile es correcto y la imagen se construye sin errores	1
El archivo compose.yaml es correcto y arranca los dos servicios	1,5
El monitor está bien configurado y detecta la caída y la recuperación	1,5
Página de estado, actualización de la web y persistencia comprobadas	1
Reenvío de puertos configurado y acceso desde el anfitrión	1
Entrega ordenada, completa y con las capturas numeradas	0,5
Total	10